GIÁO TRÌNH PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

CHƯƠNG 3: CHIẾN LƯỢC VÉT CẠN VÀ TÌM KIẾM TOÀN DIỆN (BRUTE FORCE & EXHAUSTIVE SEARCH)

1. Tổng quan Chiến lược Vét cạn (Brute Force) & Tìm kiếm Toàn diện (Exhaustive Search)

Chiến lược Vét cạn (Brute Force) là cách tiếp cận trực tiếp và tự nhiên nhất để giải quyết một bài toán, thường dựa thẳng vào phát biểu hình thức của bài toán và các định nghĩa toán học liên quan. Mặc dù hiếm khi mang lại các thuật toán có hiệu năng tiệm cận tối ưu, chiến lược Vét cạn sở hữu những ưu thế vượt bậc:

1. **Tính ứng dụng phổ quát (Wide Applicability):** Vét cạn có thể áp dụng cho hầu như mọi lớp bài toán tính toán.

2. **Tính đơn giản và chuẩn mực (Simplicity & Baseline):** Dễ hiểu, dễ cài đặt đúng, đóng vai trò làm 'chuẩn đối sánh' (benchmark) để kiểm tra tính đúng đắn của các thuật toán phức tạp hơn.

3. **Hiệu quả thực tế với kích thước nhỏ:** Khi n nhỏ, thuật toán vét cạn thường chạy nhanh hơn các thuật toán phức tạp do không có chi phí hằng số ẩn lớn.

4. **Bước đệm phát triển:** Lời giải vét cạn ban đầu thường mở đường cho các cải tiến tinh tế tiếp theo.

CHUYÊN ĐỀ 3.1. Sắp xếp chọn (Selection Sort) và Sắp xếp nổi bọt (Bubble Sort)

Lý thuyết cốt lõi & Cơ sở toán học (Mục 3.1)

Bài toán sắp xếp đóng vai trò trung tâm trong khoa học máy tính: Cho một danh sách n phần tử có thể so sánh thứ tự $A[0..n-1]$, hãy sắp xếp lại các phần tử theo thứ tự không giảm: $A[0] \le A[1] \le \ldots \le A[n-1]$.

1. Sắp xếp chọn (Selection Sort):

Bắt đầu lượt duyệt thứ i (từ $i=0$ đến $n-2$), thuật toán quét toàn bộ đoạn mảng chưa có thứ tự $A[i..n-1]$ để tìm phần tử có giá trị nhỏ nhất, sau đó hoán vị phần tử nhỏ nhất này với $A[i]$. Kết quả là sau lượt thứ i, đoạn con $A[0..i]$ chứa $i+1$ phần tử nhỏ nhất của mảng và đã được sắp xếp hoàn toàn chính xác.

2. Sắp xếp nổi bọt (Bubble Sort):

Thuật toán quét qua mảng từ trái sang phải, so sánh liên tiếp hai phần tử kề nhau $A[j]$ và $A[j+1]$. Nếu $A[j] > A[j+1]$ (nghịch thế), ta đổi chỗ chúng. Quá trình này đẩy phần tử lớn nhất dần về cuối mảng như bọt khí nổi lên mặt nước. Sau lượt thứ i, phần tử lớn thứ i đã nằm đúng vị trí cuối cùng của nó.

Các thuật toán kinh điển chuẩn hóa (Mục 3.1)

Thuật toán ALGO_3.1: SelectionSort(A[0..n - 1])

Bài toán: Sắp xếp mảng A gồm n phần tử theo thứ tự không giảm bằng phương pháp chọn phần tử cực tiểu.

Ý tưởng thiết kế: Quét mảng con A[i..n-1] tìm vị trí phần tử nhỏ nhất min_idx, rồi hoán vị A[i] với A[min_idx]. Lặp lại n - 1 lần.

```
function: SelectionSort(A, n): void
   input: A: array [0..n - 1] of orderable elements  - mảng dữ liệu đầu vào.
          n: integer (n ≥ 1)  - số lượng phần tử của mảng.
  output: A: array [0..n - 1]  - mảng ban đầu đã được sắp xếp không giảm tại chỗ.

 1  for i = 0 to n - 2 do
 2      min_idx = i  - giả định phần tử đầu đoạn chưa sắp là nhỏ nhất.
 3      for j = i + 1 to n - 1 do
 4          if A[j] < A[min_idx] then
 5              min_idx = j  - cập nhật chỉ số phần tử nhỏ nhất mới.
 6          end-if
 7      end-for
 8      swap(A[i], A[min_idx])  - đưa phần tử nhỏ nhất về vị trí i.
 9  end-for
10  return.
```

Phân tích toán học: Phép toán cơ sở là phép so sánh khóa $A[j] < A[min_idx]$.

Số phép so sánh hoàn toàn độc lập với trạng thái ban đầu của mảng:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} \left[(n-1) - (i+1) + 1 \right] = \sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \ldots + 1 = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Số lần hoán vị (swaps) tối đa: Đúng $n-1$ lần (tức $\Theta(n)$), ít hơn mọi thuật toán sắp xếp thông thường.

Độ phức tạp tính toán: {'basic_op': 'Phép so sánh khóa $A[j] < A[min_idx]$.', 'best': '$Theta(n^2)$ (số phép so sánh luôn là $fracn(n-1)2$).', 'worst': '$Theta(n^2)$.', 'avg': '$Theta(n^2)$.', 'space': '$Theta(1)$ bổ sung (sắp xếp tại chỗ in-place).'}

Nhận xét & Tối ưu hóa: SelectionSort KHÔNG ổn định (not stable). Ví dụ: dãy [2a, 2b, 1] sau bước 1 hoán vị 2a với 1 thành [1, 2b, 2a], trật tự tương đối giữa 2a và 2b bị đảo ngược.

Thuật toán ALGO_3.2: BubbleSort(A[0..n - 1])

Bài toán: Sắp xếp mảng A gồm n phần tử theo thứ tự không giảm bằng phương pháp đổi chỗ các cặp phần tử kề nhau.

Ý tưởng thiết kế: So sánh liên tiếp A[j] và A[j+1]. Nếu sai thứ tự thì đổi chỗ. Dùng cờ swapped để dừng sớm nếu mảng đã có thứ tự.

```
function: BubbleSort(A, n): void
   input: A: array [0..n - 1] of orderable elements  – mảng cần sắp xếp.
          n: integer (n ≥ 1)  – kích thước mảng.
  output: A: array [0..n - 1]  – mảng đã sắp xếp không giảm.

 1  for i = 0 to n - 2 do
 2      swapped = false  – cờ kiểm tra có phát sinh hoán vị hay không.
 3      for j = 0 to n - 2 - i do
 4          if A[j + 1] < A[j] then
 5              swap(A[j], A[j + 1])  – đổi chỗ cặp nghịch thế kề nhau.
 6              swapped = true.
 7          end-if
 8      end-for
 9      if not swapped then
10          break  – mảng đã có thứ tự hoàn chỉnh, dừng sớm.
11      end-if
12  end-for
13  return.
```

Phân tích toán học: Số phép so sánh trong trường hợp xấu nhất (mảng giảm dần):

$$C_{worst}(n) = \sum_{i=0}^{n-2}\sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2}(n-1-i) = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Số phép đổi chỗ trong trường hợp xấu nhất: Đúng bằng số phép so sánh $S_{worst}(n) = \frac{n(n-1)}{2}$.

Trường hợp tốt nhất (mảng đã sắp sẵn): Nhờ cờ swapped, thuật toán dừng sau lượt đầu tiên ($i = 0$): $C_{best}(n) = n - 1 \in \Theta(n)$.

Độ phức tạp tính toán: {'basic_op': 'Phép so sánh khóa $A[j+1] < A[j]$.', 'best': '$Theta(n)$ (khi mảng đã sắp xếp sẵn).', 'worst': '$Theta(n^2)$ (khi mảng bị đảo ngược).', 'avg': '$Theta(n^2)$ (khoảng $fracn^2 2$ phép so sánh và $fracn^2 4$ phép đổi chỗ).', 'space': '$Theta(1)$ bổ sung (in-place).'}

Nhận xét & Tối ưu hóa: BubbleSort là thuật toán sắp xếp ỔN ĐỊNH (stable) vì hai phần tử bằng nhau ($A[j] = A[j+1]$) không bao giờ bị đổi chỗ cho nhau.

Bài 3.1.1: Khái niệm thuật toán Vét cạn và Bài toán không thể giải bằng Vét cạn

> **Đề bài:** a. Hãy đưa ra một ví dụ về một thuật toán không nên được xem là một ứng dụng của cách tiếp cận vét cạn (brute-force approach).
> b. Hãy đưa ra một ví dụ về một bài toán không thể giải quyết được bằng thuật toán vét cạn.

Phân tích phương pháp giải:

Cách tiếp cận vét cạn dựa trên việc bám sát định nghĩa hình thức hoặc duyệt tuần tự mọi phần tử/phương án. Một thuật toán không phải vét cạn thường dựa trên các tính chất toán học tinh tế (như Euclid, Binary Search, Quickhull). Một bài toán không thể giải bằng vét cạn thường là bài toán trên miền vô hạn hoặc bài toán không giải được (undecidable).

Lời giải chi tiết:

a. Ví dụ thuật toán không phải Vét cạn:

• **Thuật toán Euclid tìm ước chung lớn nhất (GCD):** Thuật toán dựa trên định lý đại số $gcd(m,n) = gcd(n, mmodn)$ để rút gọn bài toán sau mỗi bước với tốc độ logarit $O(\log(min(m,n)))$. Thuật toán vét cạn cho bài toán này là duyệt thử tất cả các số nguyên từ $min(m,n)$ giảm dần về 1.

• **Tìm kiếm nhị phân (Binary Search):** Loại bỏ một nửa số phần tử sau mỗi phép so sánh trong $O(\log n)$, khác hoàn toàn với tìm kiếm tuần tự vét cạn $O(n)$.

b. Ví dụ bài toán không thể giải bằng Vét cạn:

• **Bài toán Dừng (The Halting Problem - Alan Turing, 1936):** Xác định xem một chương trình tùy ý có dừng lại trên một đầu vào cho trước hay không. Vì không gian trạng thái thực thi là vô hạn và không thể đoán trước, thuật toán vét cạn không thể duyệt hết được (nếu chương trình lặp vô tận, thuật toán vét cạn cũng sẽ chạy mãi mãi mà không bao giờ dừng để đưa ra kết luận).

• **Tìm nghiệm thực chính xác của phương trình đại số bậc cao hoặc phương trình vi phân phi tuyến:** Miền số thực là liên tục (vô hạn không đếm được), vét cạn không thể duyệt qua mọi số thực.

Bài 3.1.2: Độ phức tạp tính lũy thừa a^n theo phương pháp vét cạn

> **Đề bài:** a. Độ phức tạp thời gian của thuật toán vét cạn tính a^n là bao nhiêu theo kích thước n? Theo số bit b trong biểu diễn nhị phân của n?
> b. Nếu bạn cần tính a^n mod m với a > 1 và n là số nguyên dương rất lớn, bạn sẽ khắc phục vấn đề độ lớn cực đại của a^n như thế nào?

Phân tích phương pháp giải:

Thuật toán vét cạn tính a^n bằng cách nhân liên tiếp a với chính nó n lần: $a^n = 1 \times a \times a \ldots \times a$. Kích thước đầu vào của một số nguyên n trong khoa học máy tính được đo bằng số lượng bit nhị phân $b = \lfloor \log_2 n \rfloor + 1$.

Lời giải chi tiết:

a. Độ phức tạp thời gian:

• Theo giá trị của n: Thuật toán thực hiện chính xác $n - 1$ phép nhân số thực/số nguyên: $M(n) = n - 1 \in \Theta(n)$

• Theo số lượng bit nhị phân $b = \lfloor \log_2 n \rfloor + 1$: Vì $n \approx 2^{b-1}$, nên số phép nhân biểu diễn theo b là: $M(b) \approx 2^{b-1} \in \Theta(2^b)$

Như vậy, theo độ dài dữ liệu đầu vào b, thuật toán vét cạn tính lũy thừa thực chất là một **thuật toán có độ phức tạp hàm mũ (exponential time)**!

b. Giải pháp khắc phục độ lớn cực đại khi tính $a^n \bmod m$:

Áp dụng tính chất phân phối của phép chia lấy dư đối với phép nhân:

$$(x \cdot y) mod m = [(x mod m) \cdot (y mod m)] mod m$$

Do đó, tại MỖI BƯỚC NHÂN LIÊN TIẾP, ta lấy phần dư modulo m ngay lập tức thay vì tính lũy thừa toàn bộ rồi mới chia dư:

$$product = (product \cdot a) mod m$$

Kỹ thuật này bảo đảm giá trị trung gian không bao giờ vượt quá $a \cdot m$ (hoặc $(m-1)^2$), triệt tiêu hoàn toàn hiện tượng tràn số nguyên (integer overflow).

Bài 3.1.3: Nhận diện tiếp cận vét cạn trong các bài toán chương 2

> **Đề bài:** Đối với mỗi thuật toán trong các bài tập 4, 5 và 6 của Bài tập 2.3, hãy cho biết thuật toán đó có dựa trên cách tiếp cận vét cạn hay không?

Phân tích phương pháp giải:

Xem lại các thuật toán trong Bài tập 2.3:

• Bài 4: Tính giá trị đa thức $\sum a_i x^i$ bằng cách tính riêng từng lũy thừa x^i.

• Bài 5: Nhân hai ma trận vuông theo định nghĩa $C[i,j] = \sum A[i,k]B[k,j]$.

• Bài 6: Đếm số bit nhị phân bằng cách chia liên tiếp cho 2.

Lời giải chi tiết:

• **Bài 4 (Tính đa thức ngây thơ):** CÓ. Đây là thuật toán vét cạn kinh điển vì nó thay trực tiếp x_0 vào công thức và tính từng số hạng $a_i x_0^i$ một cách độc lập.

• **Bài 5 (Nhân ma trận theo định nghĩa):** CÓ. Thuật toán áp dụng thẳng định nghĩa đại số tuyến tính của tích ma trận thông qua 3 vòng lặp lồng nhau.

• **Bài 6 (Đếm số bit nhị phân):** KHÔNG. Thuật toán chia đôi liên tiếp kích thước số ($n \leftarrow \lfloor n/2 \rfloor$), thuộc chiến lược **Giảm để trị (Decrease-and-Conquer - Giảm theo hệ số 2)** với thời gian logarit $O(\log n)$.

Bài 3.1.4: Thiết kế thuật toán tính giá trị đa thức

> **Đề bài:** a. Thiết kế một thuật toán vét cạn để tính giá trị đa thức $p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$ tại điểm x_0 và xác định lớp hiệu quả thời gian xấu nhất của nó.
> b. Nếu thuật toán bạn thiết kế thuộc $\Theta(n^2)$, hãy thiết kế một thuật toán tuyến tính $\Theta(n)$ cho bài toán này.
> c. Có thể thiết kế một thuật toán có hiệu năng tốt hơn tuyến tính (sub-linear) cho bài toán này hay không?

Phân tích phương pháp giải:

Cách tiếp cận vét cạn tính từng số hạng $a_i x_0^i$ bằng cách nhân x_0 liên tiếp i lần. Thuật toán tuyến tính tận dụng kết quả lũy thừa trước hoặc dùng Quy tắc Horner. Một thuật toán tính đa thức bắt buộc phải đọc qua $n + 1$ hệ số đầu vào.

Lời giải chi tiết:

a. Mã giả thuật toán vét cạn:

```text
function:  BruteForcePolynomial(P, n, x0): real
  input:  P: array [0..n] of real – mảng hệ số đa thức.
          n: integer – bậc đa thức.
          x0: real – điểm cần tính.
 output:  val: real – giá trị p(x0).
1  val = P[0]
2  for i = 1 to n do
3     power = 1.0
4     for j = 1 to i do
5        power = power * x0
6     end-for
7     val = val + P[i] * power
8  end-for
9  return val.
```

Số phép nhân: $M(n) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} \in \Theta(n^2)$.

b. Thuật toán tuyến tính $\Theta(n)$:

Áp dụng Quy tắc Horner (Horner's Rule): Nhóm các số hạng theo dạng lồng nhau: $p(x) = (\dots((a_n x + a_{n-1})x + a_{n-2})x + \cdots + a_1)x + a_0$

```text
function:  HornerPolynomial(P, n, x0): real
   input:  P: array [0..n] of real; n: integer; x0: real.
  output:  val: real.
1   val = P[n]
2   for i = n - 1 down to 0 do
3       val = val * x0 + P[i]
4   end-for
5   return val.
```

Số phép nhân đúng bằng n, số phép cộng đúng bằng n, tổng thời gian đạt $\Theta(n)$.

c. Có thể đạt hiệu năng tốt hơn tuyến tính không?

KHÔNG THỂ. Cận dưới tầm thường (Trivial Lower Bound) của bài toán này là $\Omega(n)$ vì bất kỳ thuật toán nào muốn tính chính xác đa thức tổng quát đều bắt buộc phải đọc qua tối thiểu $n + 1$ hệ số $a_0, a_1, \dots, a_n$. Do đó không thể có thuật toán siêu tuyến tính $o(n)$.

Bài 3.1.5: Nhận diện cấu trúc topo mạng (Ring, Star, Mesh)

> **Đề bài:** Cho ma trận kề $A[0..n-1, 0..n-1]$ $(n > 3)$ của một mạng máy tính. Thiết kế một thuật toán vét cạn để xác định mạng có cấu trúc nào trong 3 loại: Vòng (Ring), Sao (Star), hay Lưới liên kết đầy đủ (Fully connected mesh).

Phân tích phương pháp giải:

Phân tích đặc trưng bậc của đỉnh (degree of vertices) cho từng cấu trúc:

• **Ring (Mạng vòng):** Đồ thị liên thông trong đó MỌI đỉnh đều có đúng bậc bằng 2: $\deg(v) = 2, \forall v$.

• **Star (Mạng sao):** Đồ thị có đúng 1 đỉnh trung tâm có bậc $n - 1$ và tất cả $n - 1$ đỉnh còn lại đều có bậc đúng bằng 1.

• **Fully Connected Mesh (Lưới đầy đủ K_n):** Mọi đỉnh đều có bậc đúng bằng $n - 1$.

Lời giải chi tiết:

Mã giả chuẩn hóa:

```text
function:  IdentifyTopology(A, n): string
   input:  A: array [0..n - 1, 0..n - 1] of boolean – ma trận kề.
```

```
        n: integer (n > 3) – số lượng nút mạng.
 output:  topology: string – 'Ring', 'Star', 'Mesh', hoặc 'Other'.
1  deg = array [0..n - 1] of integer = 0
2  for i = 0 to n - 1 do
3     for j = 0 to n - 1 do
4        if A[i, j] = true then
5           deg[i] = deg[i] + 1 – đếm số liên kết nối với nút i.
6        end-if
7     end-for
8  end-for
9  count_deg2 = 0; count_deg1 = 0; count_deg_n_minus_1 = 0
10 for i = 0 to n - 1 do
11    if deg[i] = 2 then count_deg2 = count_deg2 + 1 end-if
12    if deg[i] = 1 then count_deg1 = count_deg1 + 1 end-if
13    if deg[i] = n - 1 then count_deg_n_minus_1 = count_deg_n_minus_1 + 1 end-if
14 end-for
15 if count_deg_n_minus_1 = n then return 'Mesh' end-if
16 if count_deg_n_minus_1 = 1 and count_deg1 = n - 1 then return 'Star' end-if
17 if count_deg2 = n and IsConnected(A, n) then return 'Ring' end-if
18 return 'Other'.
```

Độ phức tạp thời gian: Việc duyệt ma trận kề kích thước $n \times n$ tốn đúng n^2 phép kiểm tra. Kiểm tra tính liên thông bằng BFS/DFS tốn $O(n^2)$. Do đó độ phức tạp tổng quát là $\Theta(n^2)$.

Bài 3.1.6: Lát bảng cờ 8 × 8 bằng các khối Tetromino

> **Đề bài:** Có 5 loại khối Tetromino (gồm 4 ô vuông 1 × 1): thẳng (straight - I), vuông (square - O), chữ L, chữ T, và chữ Z. Có thể lát kín bàn cờ 8 × 8 (gồm 64 ô) mà không đè lên nhau bằng:
> a. Khối thẳng? b. Khối vuông? c. Khối chữ L? d. Khối chữ T? e. Khối chữ Z?

Phân tích phương pháp giải:

Diện tích bàn cờ là $8 \times 8 = 64$ ô. Mỗi khối tetromino chiếm 4 ô, do đó cần đúng $64/4 = 16$ khối. Áp dụng phương pháp tô màu bảng cờ (Coloring argument) và tính chẵn lẻ.

Lời giải chi tiết:

• **a. Khối thẳng (Straight I-tetromino): CÓ THỂ.** Xếp 2 khối thẳng cạnh nhau thành hình chữ nhật 4 × 2 (8 ô). Ghép 8 hình chữ nhật 4 × 2 này phủ kín bàn cờ 8 × 8.

• **b. Khối vuông (Square O-tetromino): CÓ THỂ.** Khối vuông có kích thước 2×2. Xếp một lưới 4×4 gồm 16 khối vuông 2×2 phủ kín bàn cờ 8×8.

• **c. Khối chữ L (L-tetromino): CÓ THỂ.** Hai khối L ghép lại thành một hình chữ nhật 2×4 (8 ô). Ghép 8 hình chữ nhật 2×4 phủ kín toàn bộ bàn cờ.

• **d. Khối chữ T (T-tetromino): KHÔNG THỂ.** *Chứng minh bằng tô màu bàn cờ tiêu chuẩn:* Bàn cờ 8×8 có 32 ô trắng và 32 ô đen. Mỗi khối chữ T (gồm 1 ô ở thân và 3 ô ở cánh) khi đặt lên bàn cờ luôn chiếm: hoặc 3 ô trắng + 1 ô đen, hoặc 3 ô đen + 1 ô trắng (luôn chiếm số lượng ô màu lẻ: 3 và 1). Giả sử có k khối T chiếm 3 trắng + 1 đen, và $16 - k$ khối T chiếm 1 trắng + 3 đen. Tổng số ô trắng là $3k + (16 - k) = 2k + 16$. Để phủ kín bàn cờ, số ô trắng phải bằng 32: $2k + 16 = 32 \Longrightarrow 2k = 16 \Longrightarrow k = 8$. Nhưng một khối T chỉ có thể đổi màu khi lật hoặc quay, và trên lưới cờ tổng số ô trắng của 16 khối luôn có cùng tính chẵn lẻ. Chi tiết hơn: Bằng cách tô màu xen kẽ theo hàng hoặc cột, người ta chứng minh được không thể lát được.

• **e. Khối chữ Z (Z-tetromino): KHÔNG THỂ.** Tương tự lập luận tô màu bảng cờ.

Bài 3.1.7: Bài toán Chồng tiền xu giả (A Stack of Fake Coins)

> **Đề bài:** Có n chồng tiền xu trông giống hệt nhau, mỗi chồng gồm n đồng xu. Tất cả các đồng xu trong một chồng nào đó đều là tiền giả, trong khi các đồng xu ở các chồng khác là tiền thật. Xu thật nặng 10g, xu giả nặng 11g. Bạn có một chiếc cân điện tử chính xác.
> a. Thiết kế một thuật toán vét cạn để tìm ra chồng tiền giả và xác định độ phức tạp xấu nhất của nó.
> b. Số lần cân tối thiểu cần thiết để xác định chính xác chồng tiền giả là bao nhiêu?

Phân tích phương pháp giải:

Thuật toán vét cạn sẽ cân lần lượt từng chồng. Thuật toán tối ưu sẽ kết hợp số lượng đồng xu khác nhau từ mỗi chồng để giải quyết chỉ trong đúng 1 lần cân!

Lời giải chi tiết:

a. Thuật toán vét cạn:

Lần lượt lấy 1 đồng xu từ chồng thứ i ($i = 1,2, \dots, n-1$) đặt lên cân:

• Nếu đồng xu nặng 11g, chồng thứ i chính là chồng tiền giả. Dừng lại.

• Nếu đồng xu nặng 10g, tiếp tục xét chồng tiếp theo.

• Nếu sau $n - 1$ lần cân đều là 10g, chắc chắn chồng thứ n là chồng tiền giả.

Độ phức tạp xấu nhất: Cần thực hiện $n - 1$ lần cân: $W(n) = n - 1 \in \Theta(n)$.

b. Số lần cân tối thiểu: ĐÚNG 1 LẦN CÂN DUY NHẤT!

Phương pháp:

Lấy ra từ chồng thứ nhất 1 đồng xu, từ chồng thứ hai 2 đồng xu, ..., từ chồng thứ i lấy i đồng xu, ..., từ chồng thứ n lấy n đồng xu.

Tổng số đồng xu đặt lên đĩa cân là:

$$N = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Nếu tất cả đều là tiền thật (10g/đồng), tổng khối lượng chuẩn sẽ là:

$$W_{standard} = 10 \times \frac{n(n+1)}{2} = 5n(n+1) \text{ (grams)}$$

Vì mỗi đồng xu giả nặng hơn xu thật đúng 1g ($11 - 10 = 1$g), nên nếu chồng thứ k là chồng tiền giả, trên cân sẽ có đúng k đồng xu giả. Tổng khối lượng thực tế cân được W_{actual} sẽ vượt trội hơn khối lượng chuẩn một lượng đúng bằng k gam:

$$k = W_{actual} - W_{standard}$$

Giá trị số dư vượt trội k trực tiếp chỉ ra chỉ số của chồng tiền giả chỉ qua **1 lần cân duy nhất**!

Bài 3.1.8: Minh họa SelectionSort trên danh sách ký tự

> **Đề bài:** Sắp xếp danh sách các chữ cái: E, X, A, M, P, L, E theo thứ tự chữ cái bằng thuật toán SelectionSort.

Phân tích phương pháp giải:

Mảng gồm 7 ký tự: A[0..6] = [E, X, A, M, P, L, E]. Tại mỗi bước i từ 0 đến 5, tìm ký tự nhỏ nhất trong đoạn A[i..6] rồi đổi chỗ với A[i].

Lời giải chi tiết:

Quá trình thực thi từng bước:

• **Khởi tạo:** `[ E, X, A, M, P, L, E ]`

• **Bước 0 ($i = 0$):** Ký tự nhỏ nhất trong đoạn `[E..E]` là `A` tại chỉ số 2. Đổi chỗ `A[0]` (E) và `A[2]` (A):

`[ A | X, E, M, P, L, E ]`

• **Bước 1 ($i = 1$):** Ký tự nhỏ nhất trong đoạn `[X, E, M, P, L, E]` là `E` tại chỉ số 2. Đổi chỗ `A[1]` (X) và `A[2]` (E):

`[ A, E | X, M, P, L, E ]`

• **Bước 2 ($i = 2$):** Ký tự nhỏ nhất trong đoạn `[X, M, P, L, E]` là `E` tại chỉ số 6. Đổi chỗ `A[2]` (X) và `A[6]` (E):

`[ A, E, E | M, P, L, X ]`

• **Bước 3 ($i = 3$):** Ký tự nhỏ nhất trong đoạn `[M, P, L, X]` là `L` tại chỉ số 5. Đổi chỗ `A[3]` (M) và `A[5]` (L):

`[ A, E, E, L | P, M, X ]`

• **Bước 4 ($i = 4$):** Ký tự nhỏ nhất trong đoạn `[P, M, X]` là `M` tại chỉ số 5. Đổi chỗ `A[4]` (P) và `A[5]` (M):

`[ A, E, E, L, M | P, X ]`

• **Bước 5 ($i = 5$):** Ký tự nhỏ nhất trong đoạn `[P, X]` là `P` tại chỉ số 5 (chính nó). Đổi chỗ với chính nó:

`[ A, E, E, L, M, P | X ]`

Kết quả cuối cùng: `[ A, E, E, L, M, P, X ]`.

Bài 3.1.9: Tính ổn định của thuật toán SelectionSort

> **Đề bài:** Thuật toán SelectionSort có ổn định (stable) hay không? Giải thích và đưa ra phản ví dụ.

Phân tích phương pháp giải:

Một thuật toán sắp xếp là ổn định nếu nó bảo toàn trật tự tương đối giữa các phần tử có khóa bằng nhau.

Lời giải chi tiết:

Trả lời: SelectionSort KHÔNG ỔN ĐỊNH (Not Stable).

Phản ví dụ:

Xét mảng gồm 3 phần tử: `A = [ 2_a, 2_b, 1 ]` (trong đó 2_a đứng trước 2_b).

• Tại bước $i = 0$: Phần tử nhỏ nhất trong đoạn là `1` tại chỉ số 2. Thuật toán đổi chỗ `A[0]` (2_a) với `A[2]` (`1`).

• Mảng sau bước 0 trở thành: `[ 1, 2_b, 2_a ]`.

Lúc này, phần tử 2_b đã bị đẩy lên trước phần tử 2_a. Trật tự tương đối ban đầu bị thay đổi, do đó SelectionSort không ổn định.

Bài 3.1.10: Cài đặt SelectionSort cho Danh sách liên kết

> **Đề bài:** Có thể cài đặt thuật toán SelectionSort cho danh sách liên kết (linked list) với cùng độ phức tạp thời gian $\Theta(n^2)$ như phiên bản mảng hay không?

Phân tích phương pháp giải:

Trên danh sách liên kết, việc truy cập ngẫu nhiên theo chỉ số tốn $O(n)$, nhưng việc duyệt tuần tự để tìm min và thao tác đổi con trỏ/giá trị có tốn thêm thời gian không?

Lời giải chi tiết:

Trả lời: HOÀN TOÀN CÓ THỂ.

Cách cài đặt:

Tại mỗi lượt lặp, con trỏ `current` trỏ tới đầu danh sách con chưa sắp xếp. Ta dùng một con trỏ chạy quét tuần tự từ `current` đến cuối danh sách liên kết để tìm nút có giá trị nhỏ nhất `min_node` (tốn số bước đúng bằng độ dài đoạn chưa sắp).

Sau khi tìm thấy `min_node`, ta chỉ cần **hoán vị giá trị (value/data)** giữa nút `current` và nút `min_node` trong thời gian $O(1)$ (hoặc thay đổi liên kết con trỏ).

Tổng số phép so sánh vẫn là: $\sum_{i=0}^{n-2}(n-1-i) = \frac{n(n-1)}{2} \in \Theta(n^2)$

Do đó, độ phức tạp thời gian hoàn toàn giữ nguyên $\Theta(n^2)$.

Bài 3.1.11: Minh họa BubbleSort trên danh sách ký tự

> **Đề bài:** Sắp xếp danh sách các chữ cái: E, X, A, M, P, L, E theo thứ tự chữ cái bằng thuật toán BubbleSort.

Phân tích phương pháp giải:

Mảng gồm 7 ký tự: [E, X, A, M, P, L, E]. Lần lượt so sánh các cặp phần tử kề nhau A[j] và A[j+1], nếu A[j] > A[j+1] thì đổi chỗ.

Lời giải chi tiết:

Quá trình thực thi từng lượt quét (pass):

• **Khởi tạo:** `E, X, A, M, P, L, E`

• **Lượt 0:**

- E ? X: đúng trật tự -> `E, X, A, M, P, L, E`
- X > A: đổi chỗ -> `E, A, X, M, P, L, E`
- X > M: đổi chỗ -> `E, A, M, X, P, L, E`
- X > P: đổi chỗ -> `E, A, M, P, X, L, E`
- X > L: đổi chỗ -> `E, A, M, P, L, X, E`
- X > E: đổi chỗ -> `E, A, M, P, L, E, | X` (X nổi về cuối)

• **Lượt 1:**

- E > A: đổi chỗ -> `A, E, M, P, L, E, X`
- E ? M: đúng
- M ? P: đúng
- P > L: đổi chỗ -> `A, E, M, L, P, E, X`
- P > E: đổi chỗ -> `A, E, M, L, E, | P, X` (P nổi về vị trí 5)

• **Lượt 2:**

- A ? E: đúng; E ? M: đúng
- M > L: đổi chỗ -> `A, E, L, M, E, P, X`
- M > E: đổi chỗ -> `A, E, L, E, | M, P, X` (M nổi về vị trí 4)

• **Lượt 3:**

- A ? E: đúng; E ? L: đúng
- L > E: đổi chỗ -> `A, E, E, | L, M, P, X` (L nổi về vị trí 3)

• **Lượt 4:** Quét qua `A, E, E, L` không có bất kỳ phép hoán vị nào -> Dừng sớm!

Kết quả cuối cùng: `[ A, E, E, L, M, P, X ]`.

Bài 3.1.12: Chứng minh cải tiến BubbleSort với cờ dừng sớm

> **Đề bài:** a. Chứng minh rằng nếu thuật toán BubbleSort không thực hiện bất kỳ phép đổi chỗ nào trong một lượt quét, thì danh sách đã có thứ tự và thuật toán có thể dừng lại ngay lập tức.
> b. Viết mã giả của phương pháp tích hợp cải tiến này.
> c. Chứng minh rằng hiệu quả trường hợp xấu nhất của phiên bản cải tiến này vẫn là bậc hai.

Phân tích phương pháp giải:

Nếu không có phép đổi chỗ nào, tức là $A[j] \leq A[j+1]$ với mọi j, theo tính chất bắc cầu danh sách đã sắp xếp hoàn toàn.

Lời giải chi tiết:

a. Chứng minh:

Trong một lượt quét, thuật toán so sánh tất cả các cặp phần tử kề nhau $A[j]$ và $A[j+1]$ với $0 \leq j \leq n-2-i$. Nếu không có phép đổi chỗ nào phát sinh, điều đó có nghĩa là với mọi j trong phạm vi đó, ta luôn có: $A[j] \leq A[j+1]$

Theo tính chất bắc cầu của quan hệ thứ tự: $A[0] \leq A[1] \leq A[2] \leq \cdots \leq A[n-1]$. Do đó, toàn bộ danh sách đã có thứ tự tăng dần hoàn chỉnh, việc tiếp tục các lượt quét sau là dư thừa và thuật toán có thể dừng ngay lập tức.

b. Mã giả cải tiến:

```text
function:  ImprovedBubbleSort(A, n): void
   input:  A: array [0..n - 1] of orderable elements; n: integer.
  output:  A: array [0..n - 1] đã sắp xếp.
1   for i = 0 to n - 2 do
2       swapped = false
3       for j = 0 to n - 2 - i do
4           if A[j + 1] < A[j] then
5               swap(A[j], A[j + 1])
6               swapped = true
7           end-if
8       end-for
9       if not swapped then break end-if
10  end-for
11  return.
```

```

**c. Chứng minh trường hợp xấu nhất vẫn là bậc hai:**

Xét trường hợp mảng ban đầu bị đảo ngược hoàn toàn theo thứ tự giảm dần: $A = [n, n-1, \dots, 2, 1]$. Ở mỗi lượt quét, luôn có các cặp phần tử sai thứ tự và luôn phát sinh phép đổi chỗ (`swapped = true`). Thuật toán buộc phải thực hiện đầy đủ tất cả $n-1$ lượt quét. Tổng số phép so sánh là: $C_{worst}(n) = \sum_{i=0}^{n-2}(n-1-i) = \frac{n(n-1)}{2} \in \Theta(n^2)$

Do đó, trường hợp xấu nhất vẫn luôn thuộc lớp hàm bậc hai $\Theta(n^2)$.

### Bài 3.1.13: Tính ổn định của BubbleSort

> **Đề bài:** Thuật toán BubbleSort có ổn định (stable) hay không? Giải thích rõ cơ chế.

**Phân tích phương pháp giải:**

Xét điều kiện đổi chỗ: `if A[j+1] < A[j] then swap`.

**Lời giải chi tiết:**

**Trả lời: BubbleSort LÀ THUẬT TOÁN ỔN ĐỊNH (Stable).**

**Giải thích cơ chế:**

Thuật toán chỉ thực hiện phép đổi chỗ hai phần tử kề nhau khi và chỉ khi có nghịch thế nghiêm ngặt: $A[j] > A[j+1]$. Nếu hai phần tử có giá trị bằng nhau ($A[j] = A[j+1]$), điều kiện kiểm tra $A[j+1] < A[j]$ cho kết quả `false`, và phép đổi chỗ KHÔNG BAO GIỜ xảy ra. Do đó, trật tự tương đối xuất hiện ban đầu giữa các phần tử có cùng khóa luôn được duy trì nguyên vẹn trong suốt quá trình sắp xếp.

### Bài 3.1.14: Câu đố Đổi chỗ đĩa xen kẽ (Alternating Disks)

> **Đề bài:** Cho một hàng gồm 2n đĩa thuộc hai màu đen (D) và trắng (L) xếp xen kẽ: D, L, D, L, ..., D, L. Mục tiêu là đưa tất cả n đĩa trắng về bên trái và tất cả n đĩa đen về bên phải: L, L, ..., L, D, D, ..., D. Thao tác duy nhất được phép là đổi chỗ hai đĩa kề nhau. Thiết kế thuật toán giải bài toán này và xác định chính xác số phép đổi chỗ cần thực hiện.

**Phân tích phương pháp giải:**

Bài toán tương đương với việc sắp xếp mảng gồm hai giá trị (0: Trắng, 1: Đen) ban đầu là [1, 0, 1, 0, ...] bằng các phép đổi chỗ lân cận (tương tự BubbleSort).

**Lời giải chi tiết:**

**Thuật toán mô phỏng BubbleSort cải tiến:**

Ta thực hiện $n$ lượt quét từ trái sang phải trên hàng $2n$ đĩa:

• Tại lượt thứ $i$ ($i = 1, 2, \dots, n$):
```

- Nếu i lẻ: So sánh và đổi chỗ các cặp $(A[1], A[2]), (A[3], A[4]), \ldots, (A[2n-1], A[2n])$ nếu cặp đó có dạng (D, L).

- Nếu i chẵn: So sánh và đổi chỗ các cặp $(A[2], A[3]), (A[4], A[5]), \ldots, (A[2n-2], A[2n-1])$ nếu cặp đó có dạng (D, L).

Mã giả chuẩn hóa:

```text
function:  AlternatingDisks(A, n): integer
  input:  A: array [1..2n] of characters – mảng đĩa xen kẽ ['D', 'L', ...].
          n: integer – số lượng đĩa mỗi màu.
 output:  moves: integer – tổng số phép đổi chỗ hai đĩa kề nhau.
1   moves = 0
2   for i = 1 to n do
3       start_idx = (i mod 2 = 1) ? 1 : 2
4       for j = start_idx to 2 * n - 1 step 2 do
5           if A[j] = 'D' and A[j + 1] = 'L' then
6               swap(A[j], A[j + 1])
7               moves = moves + 1
8           end-if
9       end-for
10  end-for
11  return moves.
```

Tính toán chính xác số phép đổi chỗ:

Mỗi đĩa đen thứ k (vị trí ban đầu là $2k-1$) cần di chuyển về vị trí đích cuối cùng là $n+k$. Vì mỗi phép đổi chỗ hai phần tử kề nhau làm giảm số lượng cặp nghịch thế đi đúng 1 đơn vị, tổng số phép đổi chỗ bằng đúng tổng số nghịch thế ban đầu: Mỗi đĩa đen thứ k ban đầu đứng trước đúng $n-k+1$ đĩa trắng. Tổng số phép đổi chỗ tối thiểu bắt buộc là:

$$\text{Total moves} = \sum_{k=1}^{n} (n-k) = \frac{n(n-1)}{2} \text{hoặc} \frac{n(n+1)}{2} \in \Theta(n^2)$$

Với cấu hình xen kẽ $D, L, D, L, \ldots$: Đĩa đen thứ nhất đứng trước n đĩa trắng trừ 1 đĩa kề, tổng số bước di chuyển chính xác là: $\text{Moves} = \frac{n(n+1)}{2}$

CHUYÊN ĐỀ 3.2. Tìm kiếm tuần tự (Sequential Search) và Khớp chuỗi vét cạn (Brute-Force String Matching)

Lý thuyết cốt lõi & Cơ sở toán học (Mục 3.2)

1. Tìm kiếm tuần tự (Sequential Search):

So sánh lần lượt khóa tìm kiếm K với từng phần tử của mảng $A[0..n-1]$. Kỹ thuật lính canh (Sentinel): Đặt $A[n] = K$ vào cuối mảng, loại bỏ được phép kiểm tra biên $i < n$ ở mỗi bước lặp.

2. Khớp chuỗi vét cạn (Brute-Force String Matching):

Cho chuỗi văn bản $T[0..n-1]$ và mẫu $P[0..m-1]$ ($m \leq n$). Ta dịch chuyển mẫu P dọc theo T, tại mỗi vị trí i ($0 \leq i \leq n-m$), so sánh lần lượt $P[j]$ với $T[i+j]$ từ trái sang phải ($j = 0..m-1$).

Các thuật toán kinh điển chuẩn hóa (Mục 3.2)

Thuật toán ALGO_3.3: SequentialSearch2(A[0..n], K)

Bài toán: Tìm vị trí đầu tiên của khóa K trong mảng A gồm n phần tử sử dụng kỹ thuật lính canh.

Ý tưởng thiết kế: Gán A[n] = K làm lính canh. Vòng lặp while chỉ cần so sánh A[i] != K mà không cần kiểm tra chỉ số i < n.

```
function: SequentialSearchSentinel(A, n, K): integer
   input: A: array [0..n] of elements  – mảng đã cấp phát dư 1 ô tại vị trí n.
          n: integer (n ≥ 0)  – số phần tử thực tế của mảng.
          K: element  – khóa cần tìm kiếm.
  output: index: integer  – chỉ số đầu tiên mà A[index] = K, hoặc -1 nếu không tìm thấy.

 1  A[n] = K  – cài đặt lính canh tại vị trí cuối mảng.
 2  i = 0
 3  while A[i] ≠ K do
 4      i = i + 1  – duyệt qua phần tử kế tiếp mà không cần kiểm tra biên.
 5  end-while
 6  if i < n then
 7      return i  – tìm thấy khóa K trong phạm vi mảng dữ liệu thực tế.
 8  else
 9      return -1  – vòng lặp chỉ dừng khi gặp lính canh tại ô n.
10  end-if
```

Phân tích toán học: Trường hợp xấu nhất: Khóa K không có trong mảng. Vòng lặp chạy từ $i = 0$ đến n: Đúng $n+1$ phép so sánh khóa: $C_{worst}(n) = n + 1 \in \Theta(n)$.

Trường hợp tốt nhất: Khóa K nằm ngay tại $A[0]$: Đúng 1 phép so sánh: $C_{best}(n) = 1 \in \Theta(1)$.

Trường hợp trung bình (xác suất tìm thấy là p): $C_{avg}(n) = p \cdot \frac{n+1}{2} + (1-p)(n+1) = (n+1)\left(1-\frac{p}{2}\right) \in \Theta(n)$

Độ phức tạp tính toán: {'basic_op': 'Phép so sánh khóa $A[i] neK$.', 'best': '$Theta(1)$.', 'worst': '$Theta(n)$ ($n+1$ phép so sánh).', 'avg': '$Theta(n)$.', 'space': '$Theta(1)$ bổ sung.'}

Nhận xét & Tối ưu hóa: Tiết kiệm 50% số phép kiểm tra điều kiện rẽ nhánh (branching) ở cấp độ mã máy hợp ngữ.

Thuật toán ALGO_3.4: BruteForceStringMatch(T[0..n - 1], P[0..m - 1])

Bài toán: Tìm vị trí xuất hiện đầu tiên của chuỗi mẫu P độ dài m trong văn bản T độ dài n.

Ý tưởng thiết kế: Căn chỉnh mẫu P tại từng vị trí i từ 0 đến n - m. So sánh lần lượt từng ký tự từ trái sang phải.

```
function: BruteForceStringMatch(T, n, P, m): integer
   input: T: array [0..n - 1] of characters  - chuỗi văn bản.
          n: integer  - độ dài văn bản.
          P: array [0..m - 1] of characters  - chuỗi mẫu.
          m: integer (m ≤ n)  - độ dài mẫu.
  output: pos: integer  - vị trí bắt đầu của mẫu trong văn bản, hoặc -1 nếu không tìm thấy.

 1  for i = 0 to n - m do
 2      j = 0
 3      while j < m and P[j] = T[i + j] do
 4          j = j + 1
 5      end-while
 6      if j = m then
 7          return i  - tìm thấy mẫu tại vị trí i.
 8      end-if
 9  end-for
10  return -1  - không tìm thấy mẫu trong văn bản.
```

Phân tích toán học: Trường hợp xấu nhất: Tại mỗi vị trí i, thuật toán so khớp thành công $m-1$ ký tự đầu và chỉ thất bại ở ký tự thứ m (ví dụ T = "000…001" và P = "00…01"):

$$C_{worst}(n,m) = m(n-m+1) \in \Theta(m \cdot n)$$

Trường hợp tốt nhất: Mẫu khớp ngay tại đầu văn bản ($i=0$): m phép so sánh, hoặc tại mỗi vị trí ký tự đầu tiên đã không khớp: $n-m+1$ phép so sánh: $\Theta(n)$.

Trường hợp trung bình trên văn bản ngôn ngữ tự nhiên: Thường không khớp ngay từ 1-2 ký tự đầu: $C_{avg}(n,m) \in \Theta(n)$.

Độ phức tạp tính toán: {'basic_op': 'Phép so sánh ký tự $P[j] = T[i+j]$.', 'best': '$Theta(m)$ hoặc $Theta(n)$.', 'worst': '$Theta(mcdotn)$.', 'avg': '$Theta(n)$.', 'space': '$Theta(1)$ bổ sung.'}

Nhận xét & Tối ưu hóa: Trong Chương 7, thuật toán Horspool và Boyer-Moore đạt hiệu năng siêu tuyến tính $O(n/m)$ nhờ tiền xử lý mẫu.

Lời giải chi tiết 100% Toàn bộ Bài tập Mục 3.2 (11 Bài)

Bài 3.2.1: Phân tích số phép so sánh của Sequential Search có lính canh

> **Đề bài:** Tìm số phép so sánh khóa thực hiện bởi phiên bản tìm kiếm tuần tự có lính canh (SequentialSearchSentinel):
> a. Trong trường hợp xấu nhất.
> b. Trong trường hợp trung bình nếu xác suất tìm kiếm thành công là p ($0 \le p \le 1$).

Phân tích phương pháp giải:

Trong thuật toán có lính canh, phần tử K được đặt tại A[n]. Vòng lặp dừng khi A[i] = K. Nếu tìm thấy tại vị trí i (0 <= i < n), số phép so sánh là i + 1. Nếu không có trong mảng, vòng lặp dừng tại i = n, số phép so sánh là n + 1.

Lời giải chi tiết:

a. Trường hợp xấu nhất:

Xảy ra khi khóa K không xuất hiện trong mảng dữ liệu thực tế ($A[0..n-1]$), hoặc K chỉ xuất hiện ở phần tử cuối cùng $A[n-1]$. Vòng lặp `while` chỉ dừng lại khi chạm tới lính canh tại chỉ số n. Do đó, số phép so sánh khóa thực hiện trong vòng lặp chính xác là: $C_{worst}(n) = n + 1$

(Nếu tính thêm 1 phép so sánh điều kiện $i < n$ sau vòng lặp thì tổng là $n + 2$, nhưng phép toán cơ sở trong vòng lặp là $n + 1$).

b. Trường hợp trung bình:

Giả sử khi tìm kiếm thành công (xác suất p), khóa K có khả năng xuất hiện tại mỗi vị trí $0,1,\dots,n-1$ với xác suất bằng nhau là $\frac{p}{n}$. Nếu tìm thấy tại vị trí i, số phép so sánh là $i + 1$. Nếu tìm kiếm thất bại (xác suất $1 - p$), số phép so sánh là $n + 1$.

Kỳ vọng số phép so sánh trung bình là:

$$\begin{aligned} C_{avg}(n) &= \sum_{i=0}^{n-1} \frac{p}{n} \cdot (i + 1) + (1 - p)(n + 1) \\ &= \frac{p}{n} \sum_{j=1}^n j + (1 - p)(n + 1) \\ &= \frac{p}{n} \cdot \frac{n(n + 1)}{2} + (1 - p)(n + 1) \\ &= p \cdot \frac{n + 1}{2} + (1 - p)(n + 1) \\ &= (n + 1)\left[\frac{p}{2} + 1 - p\right] = (n + 1)\left(1 - \frac{p}{2}\right) \end{aligned}$$

• Nếu $p = 1$ (chắc chắn có trong mảng): $C_{avg}(n) = \frac{n+1}{2}$.

• Nếu $p = 0$ (chắc chắn không có): $C_{avg}(n) = n + 1$.

Bài 3.2.2: Cực trị của hàm kỳ vọng tìm kiếm tuần tự C_avg(n)

> **Đề bài:** Cho công thức số phép so sánh khóa trung bình của tìm kiếm tuần tự: C_avg(n) = [p(n + 1) /

2] + n(1 - p) với 0 ≤ p ≤ 1. Với n cố định, xác định giá trị của p để C_avg(n) đạt cực đại và cực tiểu.

Phân tích phương pháp giải:

Biến đổi C_avg(n) thành một hàm bậc nhất theo biến p: C_avg(p) = A*p + B, khảo sát tính đơn điệu.

Lời giải chi tiết:

Biến đổi công thức theo biến p:

$$C_{avg}(p) = p \cdot \frac{n+1}{2} + n - n \cdot p = n - p\left(n - \frac{n+1}{2}\right) = n - p \cdot \frac{n-1}{2}$$

Với $n \geq 1$, hệ số góc trước p là $-\frac{n-1}{2} \leq 0$:

• Đây là một **hàm số bậc nhất nghịch biến theo p** trên đoạn $[0, 1]$.

• **Giá trị cực đại (Maximum):** Đạt được khi p nhỏ nhất, tức $p = 0$ (tìm kiếm luôn thất bại): $C_{avg}^{max}(n) = n$

• **Giá trị cực tiểu (Minimum):** Đạt được khi p lớn nhất, tức $p = 1$ (tìm kiếm luôn thành công): $C_{avg}^{min}(n) = n - \frac{n-1}{2} = \frac{n+1}{2}$

(Trường hợp suy biến $n = 1$: $C_{avg}(p) = 1$ là hằng số với mọi p).

Bài 3.2.3: Thử nghiệm thiết bị thả rơi từ tòa nhà n tầng (Gadget Testing)

Đề bài: Một công ty muốn xác định tầng cao nhất của tòa nhà n tầng mà từ đó thả rơi một thiết bị không bị vỡ. Công ty có đúng 2 thiết bị giống hệt nhau. Nếu một cái bị vỡ, nó không thể sửa chữa và phải hoàn thành thử nghiệm với cái còn lại. Hãy thiết kế thuật toán thuộc lớp hiệu năng tốt nhất có thể để giải quyết bài toán này.

Phân tích phương pháp giải:

Nếu chỉ có 1 thiết bị, bắt buộc phải thử từ tầng 1, 2, 3... tốn O(n) lần thả. Với 2 thiết bị, thiết bị 1 có thể dùng để nhảy cóc các tầng (bước nhảy k). Khi thiết bị 1 vỡ ở tầng i*k, *thiết bị 2 sẽ dò tuần tự từ tầng (i-1)*k + 1 đến i*k - 1. Số lần thả tối đa là n/k + k. Đạt cực tiểu khi k = √n.

Lời giải chi tiết:

Chiến lược bước nhảy hằng số $k = \lceil\sqrt{n}\rceil$:

1. Lấy thiết bị 1 thả lần lượt từ các tầng cách đều nhau: $k, 2k, 3k, \ldots$ cho đến khi thiết bị 1 bị vỡ tại tầng $i \cdot k$ (hoặc đến tầng cao nhất n).

2. Khi thiết bị 1 bị vỡ tại tầng $i \cdot k$, ta biết chắc tầng tới hạn nằm trong khoảng $[(i-1)k + 1, \ldots, i \cdot k - 1]$.

3. Lấy thiết bị 2 thả tuần tự từng tầng một từ tầng $(i-1)k + 1$ đi lên. Khi thiết bị 2 vỡ tại tầng nào, tầng ngay dưới nó chính là tầng cao nhất không bị vỡ.

Độ phức tạp: Số lần thả của thiết bị 1 tối đa là $\frac{n}{k}$, số lần thả của thiết bị 2 tối đa là $k-1$. Tổng số lần thả tối đa là: $T(n)=\frac{n}{k}+k-1$

Để tối thiểu hóa tổng này, chọn $k=\sqrt{n}$. Khi đó: $T(n)=\sqrt{n}+\sqrt{n}-1=2\sqrt{n}-1\in\Theta(\sqrt{n})$

Chiến lược tối ưu tuyệt đối (Bước nhảy giảm dần):

Để số lần thả trong trường hợp xấu nhất là đồng đều ở mọi khoảng, ta chọn các tầng thả cho thiết bị 1 với khoảng cách giảm dần: $x, x-1, x-2, \dots, 1$.

Tổng số tầng phủ được là:

$$\frac{x(x+1)}{2}\geq n \Longrightarrow x\approx\sqrt{2n}\approx 1.414\sqrt{n}\in\Theta(\sqrt{n})$$

Cả hai cách đều thuộc lớp hiệu năng tối ưu tiệm cận là $\Theta(\sqrt{n})$.

Bài 3.2.4: Đếm số phép so sánh khi tìm từ GANDHI

> **Đề bài:** Xác định số phép so sánh ký tự thực hiện bởi thuật toán khớp chuỗi vét cạn khi tìm kiếm mẫu P = 'GANDHI' (m = 6) trong chuỗi văn bản T = 'THERE_IS_MORE_TO_LIFE_THAN_INCREASING_ITS_SPEED' (n = 47 ký tự).

Phân tích phương pháp giải:

Duyệt qua từng vị trí i từ 0 đến 47 - 6 = 41. Tại mỗi vị trí i, so sánh T[i] với P[0] = 'G'. Nếu T[i] != 'G', tốn 1 phép so sánh và chuyển sang i + 1. Nếu T[i] == 'G', tiếp tục so sánh các ký tự tiếp theo.

Lời giải chi tiết:

Mẫu $P=$ "GANDHI", độ dài $m=6$. Ký tự đầu tiên là `'G'`.

Văn bản T có $n=47$ ký tự. Số vị trí căn chỉnh cần xét là $n-m+1=47-6+1=42$ vị trí (từ $i=$ 0 đến 41).

Xét sự xuất hiện của ký tự `'G'` trong chuỗi T:

Văn bản: `T H E R E _ I S _ M O R E _ T O _ L I F E _ T H A N _ I N C R E A S I N G _ I T S _ S P E E D`

Ký tự `'G'` chỉ xuất hiện duy nhất **1 lần** tại từ `INCREASING` (vị trí chỉ số 39).

• Tại 41 vị trí $i\neq 39$: Ký tự đầu tiên $T[i]\neq$ 'G', thuật toán chỉ tốn đúng 1 phép so sánh rồi thất bại ngay. Tổng cộng: $41\times 1=41$ phép so sánh.

• Tại vị trí $i=39$ (chữ `'G'` của `INCREASING`):

- So sánh 1: $T[39]=$ 'G' $==P[0]=$ 'G' (Trùng khớp!)
- So sánh 2: $T[40]=$ '_' $\neq P[1]=$ 'A' (Không khớp! Dừng lại).

Vị trí này tốn đúng 2 phép so sánh.

Tổng số phép so sánh ký tự thực hiện:

$$\text{Total comparisons}=41+2=43 \text{ phép so sánh.}$$

Bài 3.2.5: Đếm phép so sánh trên văn bản nhị phân 1.000 số 0

> **Đề bài:** Có bao nhiêu phép so sánh ký tự (cả thành công và thất bại) được thực hiện bởi thuật toán khớp chuỗi vét cạn khi tìm kiếm các mẫu sau trong văn bản nhị phân gồm 1.000 số 0 (n = 1.000)?
> a. 00001 (m = 5)
> b. 10000 (m = 5)
> c. 01010 (m = 5)

Phân tích phương pháp giải:

Văn bản T toàn số 0: T[i] = '0' với mọi 0 <= i < 1000. Số vị trí căn chỉnh là n - m + 1 = 1000 - 5 + 1 = 996 vị trí (i = 0..995).

Lời giải chi tiết:

Số vị trí căn chỉnh là $1000 - 5 + 1 = 996$ vị trí ($i = 0,1, \dots ,995$).

a. Mẫu P = '00001':

Tại mỗi vị trí i, 4 ký tự đầu của mẫu đều là `'0'`, khớp hoàn hảo với 4 ký tự của văn bản ($T[i..i + 3] = "0000"$). Đến ký tự thứ 5: $P[4] = '1' \neq T[i + 4] = '0'$ (thất bại). Như vậy tại mỗi vị trí trong 996 vị trí, thuật toán thực hiện đúng 5 phép so sánh (4 thành công, 1 thất bại):

$$\text{Total} = 996 \times 5 = 4.980 \text{ phép so sánh.}$$

b. Mẫu P = '10000':

Ký tự đầu tiên $P[0] = '1'$. Tại mọi vị trí i, $T[i] = '0' \neq '1'$, thất bại ngay ở phép so sánh đầu tiên. Mỗi vị trí chỉ tốn đúng 1 phép so sánh thất bại:

$$\text{Total} = 996 \times 1 = 996 \text{ phép so sánh.}$$

c. Mẫu P = '01010':

Tại mỗi vị trí i:

• Phép so sánh 1: $P[0] = '0' == T[i] = '0'$ (Thành công).

• Phép so sánh 2: $P[1] = '1' \neq T[i + 1] = '0'$ (Thất bại, dừng lại).

Mỗi vị trí tốn đúng 2 phép so sánh:

$$\text{Total} = 996 \times 2 = 1.992 \text{ phép so sánh.}$$

Bài 3.2.6: Trường hợp xấu nhất của Khớp chuỗi vét cạn

> **Đề bài:** Đưa ra ví dụ về một văn bản độ dài n và một mẫu độ dài m cấu thành trường hợp xấu nhất cho thuật toán khớp chuỗi vét cạn. Chính xác có bao nhiêu phép so sánh ký tự được thực hiện?

Phân tích phương pháp giải:

Trường hợp xấu nhất xảy ra khi tại mọi vị trí i, mẫu khớp m - 1 ký tự đầu và chỉ thất bại ở ký tự cuối cùng.

Lời giải chi tiết:

Ví dụ trường hợp xấu nhất:

• Văn bản T gồm n ký tự toàn chữ `'A'`: T = "AAA...A" (độ dài n).

• Mẫu P gồm $m-1$ chữ `'A'` và kết thúc bằng chữ `'B'`: P = "AAA...AB" (độ dài m).

Số phép so sánh chính xác:

Số vị trí căn chỉnh là $n-m+1$.

Tại mỗi vị trí i ($0 \le i \le n-m$):

• Thuật toán so khớp thành công $m-1$ ký tự `'A'` đầu tiên.

• Đến ký tự cuối cùng, so sánh $P[m-1]$ = 'B' với $T[i+m-1]$ = 'A' bị thất bại.

Như vậy tại MỖI vị trí trong số $n-m+1$ vị trí, thuật toán thực hiện đúng m phép so sánh.

Tổng số phép so sánh chính xác tuyệt đối là:

$$C_{worst}(n,m) = m(n-m+1)$$

Bài 3.2.7: So sánh từ phải sang trái trong khớp chuỗi

> **Đề bài:** Khi giải bài toán so khớp chuỗi, việc so sánh các ký tự của mẫu và văn bản từ phải sang trái thay vì từ trái sang phải có đem lại lợi thế nào không?

Phân tích phương pháp giải:

Nếu chỉ dịch chuyển 1 vị trí mỗi lần (vét cạn thuần túy), độ phức tạp lý thuyết là như nhau. Tuy nhiên, nếu so sánh từ phải sang trái, khi gặp ký tự không khớp, ta có thể biết được ký tự văn bản đó có xuất hiện trong mẫu hay không để nhảy cóc!

Lời giải chi tiết:

Trả lời: CÓ LỢI THẾ CỰC KỲ LỚN (Tiền đề của thuật toán Horspool và Boyer-Moore).

Giải thích chi tiết:

Nếu giữ nguyên cơ chế dịch chuyển 1 vị trí của vét cạn, thì việc so sánh từ phải sang trái không làm thay đổi độ phức tạp xấu nhất $O(m \cdot n)$.

TUY NHIÊN, việc so sánh từ phải sang trái cho phép ta thực hiện **bước nhảy dịch chuyển lớn hơn 1** khi phát hiện không khớp:

• Giả sử ta so sánh ký tự cuối cùng của mẫu $P[m-1]$ với ký tự văn bản $T[i+m-1]$.

• Nếu ký tự $c = T[i+m-1]$ hoàn toàn không xuất hiện ở bất kỳ đâu trong mẫu P, thì không một vị trí căn chỉnh nào trong phạm vi m ký tự này có thể khớp!

• Do đó, ta có thể **nhảy cóc nguyên cả cửa sổ sang phải m vị trí** ngay lập tức!

Ý tưởng đột phá này chính là cơ sở của hai thuật toán so khớp chuỗi nhanh nhất trong lịch sử: **Thuật toán Horspool** và **Thuật toán Boyer-Moore** (khảo sát chi tiết ở Chương 7).

Bài 3.2.8: Đếm số chuỗi con bắt đầu bằng A và kết thúc bằng B

> **Đề bài:** Cho chuỗi văn bản T. Đếm số lượng chuỗi con bắt đầu bằng chữ cái 'A' và kết thúc bằng chữ cái 'B'. (Ví dụ chuỗi 'CABAAXBYA' có 4 chuỗi con như vậy: 'AB', 'AAXB', 'AXB', 'AB').
> a. Thiết kế thuật toán vét cạn và xác định độ phức tạp.
> b. Thiết kế một thuật toán hiệu quả hơn.

Phân tích phương pháp giải:

Vét cạn: Duyệt mọi cặp (i, j) với i < j, nếu T[i]=='A' và T[j]=='B' thì tăng biến đếm.

Tối ưu: Quét một lượt từ trái sang phải, đếm số lượng chữ 'A' đã gặp. Mỗi khi gặp chữ 'B', số chuỗi con kết thúc tại B này đúng bằng số chữ 'A' đứng trước nó!

Lời giải chi tiết:

a. Thuật toán vét cạn $\Theta(n^2)$:

```text
function:  BruteForceCountAB(T, n): integer
   input:  T: array [0..n - 1] of characters; n: integer.
  output:  count: integer – số chuỗi con bắt đầu bằng A kết thúc bằng B.
1   count = 0
2   for i = 0 to n - 2 do
3       if T[i] = 'A' then
4           for j = i + 1 to n - 1 do
5               if T[j] = 'B' then
6                   count = count + 1
7               end-if
8           end-for
9       end-if
10  end-for
11  return count.
```

Độ phức tạp thời gian xấu nhất: $\Theta(n^2)$ (ví dụ chuỗi gồm toàn chữ A theo sau bởi toàn chữ B).

b. Thuật toán tối ưu tuyến tính $\Theta(n)$:

Nhận xét: Mỗi khi gặp một ký tự `'B'`, ký tự này có thể kết hợp với TẤT CẢ các ký tự `'A'` đã xuất hiện trước nó để tạo thành một chuỗi con hợp lệ.

```text
function:  OptimalCountAB(T, n): integer
```

```
  input:  T: array [0..n - 1] of characters; n: integer.
 output:  total_count: integer.
1   total_count = 0
2   count_A = 0 – số lượng ký tự 'A' đã duyệt qua.
3   for i = 0 to n - 1 do
4       if T[i] = 'A' then
5           count_A = count_A + 1 – tích lũy số lượng chữ A.
6       else if T[i] = 'B' then
7           total_count = total_count + count_A – cộng dồn số chuỗi con kết thúc tại B này.
8       end-if
9   end-for
10  return total_count.
```

Độ phức tạp: Chỉ cần một lượt quét tuyến tính duy nhất qua chuỗi: Thời gian $\Theta(n)$, bộ nhớ $\Theta(1)$.

Bài 3.2.9: Thiết kế chương trình trực quan hóa khớp chuỗi vét cạn

> **Đề bài:** Mô tả thuật toán và nguyên lý thiết kế cho một chương trình trực quan hóa (visualization program) thuật toán khớp chuỗi vét cạn trên máy tính.

Phân tích phương pháp giải:

Trực quan hóa đòi hỏi biểu diễn trạng thái cửa sổ trượt, hiển thị vị trí căn chỉnh i, làm nổi bật ký tự đang so sánh, phân biệt trạng thái trùng khớp (xanh) và không khớp (đỏ).

Lời giải chi tiết:

Cấu trúc dữ liệu và Trạng thái hiển thị:

• Văn bản T cố định trên dòng trên cùng. Mẫu P hiển thị ở dòng dưới, dịch chuyển tương ứng theo chỉ số i.

• Tại mỗi bước i ($0 \leq i \leq n - m$):

1. Vẽ mẫu P thụt đầu dòng đúng i ký tự so với văn bản T.

2. Lặp con trỏ j từ 0 đến $m - 1$:

- Đổi màu ô $T[i + j]$ và $P[j]$ sang màu vàng (đang so sánh).

- Nếu $T[i + j] == P[j]$: Đổi màu sang XANH LÁ (khớp), phát âm thanh beep ngắn, tăng j.

- Nếu $T[i + j] \neq P[j]$: Đổi màu sang ĐỎ (không khớp), dừng lặp j, chuyển sang bước $i + 1$.

3. Nếu $j == m$: Đánh dấu toàn bộ đoạn $T[i..i + m - 1]$ viền nhấp nháy báo hiệu TÌM THẤY MẪU.

Bài 3.2.10: Giải câu đố Tìm từ (Word Find Puzzle)

> **Đề bài:** Thiết kế thuật toán vét cạn giải câu đố 'Tìm từ' (Word Search): Cho bảng chữ cái kích thước R × C và danh sách từ khóa. Một từ có thể nằm theo 8 hướng: ngang (trái/phải), dọc (lên/xuống), và 4 đường chéo 45 độ.

Phân tích phương pháp giải:

Với mỗi từ khóa W trong danh sách, quét qua mọi ô (r, c) trong bảng. Nếu ô (r, c) khớp ký tự đầu tiên W[0], thử mở rộng theo 8 hướng véc-tơ.

Lời giải chi tiết:

Mã giả chuẩn hóa thuật toán giải Word Find:

```text
function:  WordSearch(Board, R, C, Word, m): list of coordinates
  input:  Board: 2D-array [0..R - 1, 0..C - 1] of characters.
          R, C: integers – kích thước bảng chữ cái.
          Word: array [0..m - 1] of characters – từ cần tìm.
          m: integer – độ dài của từ.
 output:  Path: list of (row, col) – tọa độ các ô của từ tìm thấy, hoặc ∅.
1   dx = [ 0,  0,  1, -1,  1,  1, -1, -1 ] – 8 hướng di chuyển.
2   dy = [ 1, -1,  0,  0,  1, -1,  1, -1 ]
3   for r = 0 to R - 1 do
4      for c = 0 to C - 1 do
5         if Board[r, c] = Word[0] then
6            for dir = 0 to 7 do – thử cả 8 hướng xung quanh ô (r, c)
7               matched = true
8               curr_r = r; curr_c = c
9               for k = 1 to m - 1 do
10                 curr_r = curr_r + dx[dir]
11                 curr_c = curr_c + dy[dir]
12                 if curr_r < 0 or curr_r ≥ R or curr_c < 0 or curr_c ≥ C then
13                    matched = false; break – vượt biên bảng.
14                 end-if
15                 if Board[curr_r, curr_c] ≠ Word[k] then
16                    matched = false; break – ký tự không khớp.
17                 end-if
```

```
18              end-for
19              if matched then return LấyTọaĐộ(r, c, dir, m) end-if
20            end-for
21          end-if
22        end-for
23  end-for
24  return ∅ – không tìm thấy từ trong bảng.
```

Độ phức tạp: Với mỗi từ độ dài m, xét $R \times C$ ô, mỗi ô thử 8 hướng, mỗi hướng so khớp tối đa m ký tự: $T(R, C, m) \in O(R \cdot C \cdot m)$

Bài 3.2.11: Thuật toán máy chơi game Bắn tàu (Battleship Game)

> **Đề bài:** Thiết kế thuật toán dựa trên so khớp mẫu vét cạn để máy tính chơi trò Bắn tàu (Battleship) trên bàn cờ 10 × 10.

Phân tích phương pháp giải:

Máy tính cần duy trì ma trận trạng thái các ô đã bắn (Trúng, Trượt, Chưa bắn) và tập hợp các tàu đối phương còn sống (tàu 2 ô, 3 ô, 4 ô, 5 ô). Sử dụng chiến lược vét cạn xác suất (Hunt & Target).

Lời giải chi tiết:

Chiến lược 2 chế độ (Hunt Mode & Target Mode):

1. **Chế độ Săn tìm (Hunt Mode):** Khi chưa bắn trúng tàu nào:

• Đặt giả định thử từng vị trí khả dĩ của tất cả các tàu còn sống trên bàn cờ (mỗi tàu có thể đặt ngang hoặc dọc tại các ô chưa bắn).

• Với mỗi ô (r, c) chưa bắn, tính điểm xác suất $P[r, c]$ bằng tổng số cách đặt hợp lệ của các con tàu phủ qua ô đó.

• Chọn ô (r, c) có xác suất $P[r, c]$ cao nhất để bắn (kết hợp mô hình bàn cờ caro chẵn/lẻ vì tàu nhỏ nhất có kích thước 2).

2. **Chế độ Tiêu diệt (Target Mode):** Ngay khi bắn trúng một ô (r, c) mà tàu chưa chìm:

• Chuyển sang bắn 4 ô lân cận trực tiếp: $(r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)$.

• Khi bắn trúng ô thứ hai, xác định được hướng của tàu (ngang hay dọc), tiếp tục bắn dọc theo hai đầu của đường thẳng đó cho đến khi tàu chìm hoàn toàn.

CHUYÊN ĐỀ 3.3. Bài toán Cặp điểm gần nhất và Bao lồi vét cạn (Closest-Pair & Convex-Hull)

Lý thuyết cốt lõi & Cơ sở toán học (Mục 3.3)

1. Cặp điểm gần nhất (Closest-Pair Problem):

Cho n điểm $P_1, \dots, P_n$ trong mặt phẳng với tọa độ $P_i = (x_i, y_i)$. Tìm cặp điểm có khoảng cách Euclidean nhỏ nhất: $d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$

Vét cạn: Duyệt qua tất cả $\frac{n(n-1)}{2}$ cặp điểm, tính khoảng cách và cập nhật cực tiểu.

2. Bao lồi (Convex-Hull Problem):

Một tập hợp điểm S trong mặt phẳng được gọi là lồi nếu với mọi cặp điểm $P, Q \in S$, đoạn thẳng nối PQ nằm trọn vẹn trong S. Bao lồi của tập n điểm là đa giác lồi nhỏ nhất chứa tất cả các điểm đó. Đặc trưng vét cạn: Đoạn thẳng nối hai điểm P_i, P_j là một cạnh của bao lồi khi và chỉ khi **tất cả các điểm còn lại** đều nằm về cùng một phía của đường thẳng đi qua P_i, P_j (tức là dấu của biểu thức đường thẳng $ax + by - c$ cùng dấu).

Các thuật toán kinh điển chuẩn hóa (Mục 3.3)

Thuật toán ALGO_3.5: BruteForceClosestPair(P)

Bài toán: Tìm khoảng cách nhỏ nhất giữa hai điểm bất kỳ trong tập n điểm trong mặt phẳng hai chiều.

Ý tưởng thiết kế: Tính bình phương khoảng cách dx² + dy² cho mọi cặp điểm (P[i], P[j]) với i < j để cập nhật giá trị nhỏ nhất, tránh tính hàm sqrt trong vòng lặp.

```
function: BruteForceClosestPair(P, n): real
   input: P: array [1..n] of points  – tập n điểm với tọa độ (x, y).
          n: integer (n ≥ 2)  – số lượng điểm.
  output: dmin: real  – khoảng cách nhỏ nhất giữa hai điểm.

 1  d_sqr = ∞  – khởi tạo bình phương khoảng cách nhỏ nhất là vô cực.
 2  for i = 1 to n - 1 do
 3      for j = i + 1 to n do
 4          dx = P[i].x - P[j].x
 5          dy = P[i].y - P[j].y
 6          dist_sqr = dx * dx + dy * dy  – tính bình phương khoảng cách.
 7          if dist_sqr < d_sqr then
 8              d_sqr = dist_sqr  – cập nhật kỷ lục khoảng cách nhỏ nhất.
 9          end-if
10      end-for
11  end-for
```

```
12  return √d_sqr  – chỉ khai căn bậc hai đúng một lần khi trả về.
```

Phân tích toán học: Số cặp điểm được xét đúng bằng số tổ hợp chập 2 của n phần tử: $C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \frac{n(n-1)}{2} \in \Theta(n^2)$

Mỗi cặp điểm tốn 2 phép trừ, 2 phép nhân, 1 phép cộng và 1 phép so sánh: Tổng thời gian là $\Theta(n^2)$.

Độ phức tạp tính toán: {'basic_op': 'Phép tính khoảng cách bình phương $dx^2 + dy^2$.', 'best': '$Theta(n^2)$.', 'worst': '$Theta(n^2)$.', 'avg': '$Theta(n^2)$.', 'space': '$Theta(1)$ bổ sung.'}

Nhận xét & Tối ưu hóa: Trong Chương 5, thuật toán Chia để trị giải quyết bài toán này trong thời gian tối ưu tiệm cận $O(n\log n)$.

Thuật toán ALGO_3.6: BruteForceConvexHull(P)

Bài toán: Xác định tập các đoạn thẳng tạo nên biên của bao lồi cho tập n điểm.

Ý tưởng thiết kế: Với mỗi cặp điểm (Pi, Pj), lập phương trình đường thẳng ax + by = c. Kiểm tra xem tất cả n - 2 điểm còn lại có cùng nằm về một nửa mặt phẳng hay không.

```
function: BruteForceConvexHull(P, n): set of segments
   input: P: array [1..n] of points  – tập n điểm trong mặt phẳng.
          n: integer (n ≥ 3)  – số lượng điểm.
  output: Segments: set of segments  – tập các cạnh của đa giác bao lồi.

 1  Segments = ∅
 2  for i = 1 to n - 1 do
 3      for j = i + 1 to n do
 4          a = P[j].y - P[i].y
 5          b = P[i].x - P[j].x
 6          c = P[i].x * P[j].y - P[i].y * P[j].x
 7          all_on_one_side = true
 8          sign = 0
 9          for k = 1 to n do
10              if k ≠ i and k ≠ j then
11                  val = a * P[k].x + b * P[k].y - c
12                  if val ≠ 0 then
13                      curr_sign = (val > 0) ? 1 : -1
14                      if sign = 0 then sign = curr_sign
15                      else if curr_sign ≠ sign then
16                          all_on_one_side = false
17                          break
18                      end-if
19                  end-if
20              end-if
21          end-for
```

```
        if all_on_one_side then
            Segments = Segments ∪ { Segment(P[i], P[j]) }
        end-if
    end-for
end-for
return Segments.
```

Phân tích toán học: Có $\frac{n(n-1)}{2}$ cặp điểm (P_i, P_j). Với mỗi cặp điểm, duyệt qua $n-2$ điểm còn lại để kiểm tra nửa mặt phẳng:

$$T(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} (n-2) = \frac{n(n-1)}{2}(n-2) = \frac{n(n-1)(n-2)}{2} \in \Theta(n^3)$$

Độ phức tạp tính toán: {'basic_op': 'Phép tính giá trị đại số $acdotx_k + bcdoty_k - c$.', 'best': '$O(n^2)$ (khi các cặp điểm bị loại sớm).', 'worst': '$Theta(n^3)$.', 'avg': '$O(n^3)$.', 'space': '$O(n)$ lưu trữ tập cạnh bao lồi.'}

Nhận xét & Tối ưu hóa: Độ phức tạp $O(n^3)$ rất chậm. Thuật toán Quickhull (Chương 5) hoặc Graham scan giải quyết trong $O(nlogn)$.

Lời giải chi tiết 100% Toàn bộ Bài tập Mục 3.3 (12 Bài)

Bài 3.3.1: Ước lượng mức tăng tốc khi loại bỏ hàm sqrt trong Cặp điểm gần nhất

> **Đề bài:** Giả sử phép tính căn bậc hai sqrt tốn thời gian gấp khoảng 10 lần so với mỗi phép toán số học khác trong vòng lặp trong cùng của BruteForceClosestPair (phép trừ dx, dy, phép nhân bình phương, phép cộng và so sánh). Hãy ước lượng thuật toán sẽ chạy nhanh hơn bao nhiêu lần sau khi áp dụng cải tiến tránh tính sqrt trong vòng lặp?

Phân tích phương pháp giải:

Đếm số lượng phép toán trong vòng lặp trong cùng ở phiên bản gốc và phiên bản cải tiến.

Lời giải chi tiết:

Trong vòng lặp trong cùng của thuật toán gốc (có tính `sqrt`):

Mỗi cặp điểm cần thực hiện: 2 phép trừ (dx, dy), 2 phép nhân (dx^2, dy^2), 1 phép cộng, 1 phép khai căn `sqrt`, và 1 phép so sánh với `dmin`.

Giả sử mỗi phép toán cơ bản $(+, -, \times, <)$ tốn 1 đơn vị thời gian t. Phép tính `sqrt` tốn $10t$.

• **Thời gian phiên bản gốc:**

$$T_{original} = 2t + 2t + 1t + 10t + 1t = 16t$$

• **Thời gian phiên bản cải tiến (chỉ tính khoảng cách bình phương $dx^2 + dy^2$):**

Không còn hàm `sqrt` trong vòng lặp:

$$T_{improved} = 2t + 2t + 1t + 0 + 1t = 6t$$

(Phép tính `sqrt` duy nhất được dời ra ngoài vòng lặp khi trả về kết quả, chi phí chỉ là $10t$ một lần duy nhất).

Tỷ lệ tăng tốc:

$$\text{Speedup} = \frac{T_{original}}{T_{improved}} = \frac{16t}{6t} \approx 2.67 \text{ lần}$$

Thuật toán chạy nhanh hơn khoảng **2.67 lần** (tăng tốc hơn 160%) chỉ nhờ một tối ưu hóa toán học đơn giản!

Bài 3.3.2: Thuật toán hiệu quả cho Cặp điểm gần nhất trên trục số thực 1D

> **Đề bài:** Bạn có thể thiết kế một thuật toán hiệu quả hơn thuật toán vét cạn $\Theta(n^2)$ để giải bài toán cặp điểm gần nhất cho n điểm $x_1, x_2, \dots, x_n$ trên trục số thực một chiều hay không?

Phân tích phương pháp giải:

Trên trục 1 chiều, nếu các điểm đã có thứ tự, điểm gần nhất với x_i chỉ có thể là x_{i-1} hoặc x_{i+1}.

Lời giải chi tiết:

Trả lời: HOÀN TOÀN CÓ THỂ với thời gian $O(n \log n)$.

Thuật toán dựa trên Tiền sắp xếp (Presorting - Biến đổi để trị):

1. Sắp xếp n điểm số thực theo thứ tự tăng dần: $x'_1 \le x'_2 \le \cdots \le x'_n$ bằng thuật toán Mergesort hoặc Heapsort tốn thời gian $O(n\log n)$.

2. Sau khi đã sắp xếp, hai điểm gần nhau nhất BẮT BUỘC phải là hai điểm đứng kề nhau trong dãy đã sắp!

3. Quét một lượt từ trái sang phải, tính khoảng cách giữa các cặp kề nhau $x'_{i+1} - x'_i$ với $i = 1, \dots, n-1$ và tìm giá trị nhỏ nhất (tốn $O(n)$).

Mã giả chuẩn hóa:

```text
function:  ClosestPair1D(X, n): real
   input:  X: array [1..n] of real numbers; n: integer (n ≥ 2).
  output:  dmin: real – khoảng cách nhỏ nhất.
1   Mergesort(X, n) – sắp xếp mảng tăng dần trong O(n log n).
2   dmin = X[2] - X[1]
3   for i = 2 to n - 1 do
4       diff = X[i + 1] - X[i]
5       if diff < dmin then dmin = diff end-if
```

6 end-for

7 return dmin.

```

**Độ phức tạp:** $O(n\log n) + O(n) = \Theta(n\log n)$, vượt trội hoàn toàn so với $\Theta(n^2)$ của vét cạn.

### Bài 3.3.3: Vị trí đặt bưu điện tối ưu cho n ngôi làng

> **Đề bài:** Cho $n$ ngôi làng nằm trên một con đường thẳng có tọa độ $x_1 < x_2 < \cdots < x_n$. Cần xây một bưu điện tại một trong các ngôi làng đó.
> a. Thiết kế thuật toán tìm vị trí đặt bưu điện sao cho tổng khoảng cách (hoặc khoảng cách trung bình) đến tất cả các ngôi làng là nhỏ nhất.
> b. Thiết kế thuật toán tìm vị trí đặt bưu điện sao cho khoảng cách lớn nhất từ một ngôi làng đến bưu điện là nhỏ nhất.

**Phân tích phương pháp giải:**

Bài toán tối ưu hóa 1 chiều:

a. Min tổng khoảng cách ∑ |x_i - p|: Nghiệm tối ưu của hàm giá trị tuyệt đối là Trung vị (Median).

b. Min khoảng cách cực đại max |x_i - p|: Nghiệm tối ưu là điểm chính giữa đoạn [x_1, x_n].

**Lời giải chi tiết:**

**a. Tối thiểu hóa khoảng cách trung bình (Tổng khoảng cách $\sum_{i=1}^n |x_i - p|$ nhỏ nhất):**

Về mặt toán học, hàm $f(p) = \sum_{i=1}^{n} |x_i - p|$ đạt cực tiểu tại **Trung vị (Median)** của tập dữ liệu:

• Nếu $n$ lẻ: Đặt bưu điện tại ngôi làng ở chính giữa: làng thứ $m = \frac{n+1}{2}$, tức tọa độ $x_{(n+1)/2}$.

• Nếu $n$ chẵn: Bưu điện có thể đặt tại bất kỳ ngôi làng nào giữa làng thứ $n/2$ và làng thứ $n/2 + 1$ (cả hai đều cho tổng khoảng cách bằng nhau).

**Độ phức tạp:** Vì các ngôi làng đã được sắp xếp sẵn ($x_1 < x_2 < \cdots < x_n$), ta chỉ cần chọn chỉ số $\lfloor (n+1)/2 \rfloor$ trong thời gian $O(1)$!

**b. Tối thiểu hóa khoảng cách cực đại ($\max_i |x_i - p|$ nhỏ nhất):**

Khoảng cách cực đại luôn được quyết định bởi hai ngôi làng ở xa nhất: làng đầu tiên $x_1$ và làng cuối cùng $x_n$. Điểm lý tưởng là trung điểm của đoạn đường: $x_{mid} = \frac{x_1 + x_n}{2}$.

Vì bưu điện bắt buộc phải đặt tại MỘT TRONG CÁC NGÔI LÀNG, ta chỉ cần tìm ngôi làng $x_k$ có tọa độ gần $x_{mid}$ nhất:

Áp dụng tìm kiếm nhị phân tìm ngôi làng gần $\frac{x_1 + x_n}{2}$ nhất trong thời gian $O(\log n)$.

### Bài 3.3.4: Khoảng cách Manhattan và Hình học khoảng cách

> **Đề bài:** Khoảng cách Manhattan giữa hai điểm $p_1(x_1, y_1)$ và $p_2(x_2, y_2)$ được định nghĩa là:
> d_M(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2|.
```

a. Chứng minh d_M thỏa mãn 3 tiên đề của một hàm khoảng cách (metric space).

b. Vẽ tập hợp tất cả các điểm có khoảng cách Manhattan tới gốc (0, 0) bằng 1. So sánh với khoảng cách Euclidean.

c. Đúng hay sai: Nghiệm của bài toán cặp điểm gần nhất không phụ thuộc vào việc sử dụng metric d_E (Euclidean) hay d_M (Manhattan)?

Phân tích phương pháp giải:

3 tiên đề metric: Không âm & đồng nhất; Đối xứng; Bất đẳng thức tam giác.

Lời giải chi tiết:

a. Chứng minh 3 tiên đề:

1. *Tính không âm và đồng nhất:* Vì $|a| \geq 0$ nên $d_M(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2| \geq 0$. Đẳng thức xảy ra khi $|x_1 - x_2| = 0$ và $|y_1 - y_2| = 0 \Leftrightarrow x_1 = x_2$ và $y_1 = y_2 \Leftrightarrow p_1 = p_2$.

2. *Tính đối xứng:* $d_M(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2| = |x_2 - x_1| + |y_2 - y_1| = d_M(p_2, p_1)$.

3. *Bất đẳng thức tam giác:* Với điểm $p_3(x_3, y_3)$, áp dụng $|a - b| \leq |a - c| + |c - b|$:

$$|x_1 - x_2| \leq |x_1 - x_3| + |x_3 - x_2|, |y_1 - y_2| \leq |y_1 - y_3| + |y_3 - y_2|$$

Cộng hai vế ta được: $d_M(p_1, p_2) \leq d_M(p_1, p_3) + d_M(p_3, p_2)$. (Tiên đề được chứng minh!)

b. Tập các điểm có khoảng cách tới gốc $(0, 0)$ bằng 1:

• Với Manhattan: $|x| + |y| = 1$. Đây là một **hình vuông nghiêng 45 độ (hình thoi)** có 4 đỉnh tại $(1,0), (0,1), (-1,0), (0,-1)$.

• Với Euclidean: $\sqrt{x^2 + y^2} = 1 \Leftrightarrow x^2 + y^2 = 1$. Đây là **đường tròn đơn vị** bán kính $R = 1$.

c. Đúng hay sai? Trả lời: SAI (FALSE).

Nghiệm cặp điểm gần nhất HOÀN TOÀN CÓ THỂ THAY ĐỔI tùy thuộc vào metric sử dụng!

Phản ví dụ: Xét 3 điểm $A(0,0)$, $B(0.6,0.6)$, và $C(1,0)$:

• Khoảng cách Euclidean:

- $d_E(A, B) = \sqrt{0.6^2 + 0.6^2} = \sqrt{0.72} \approx 0.8485$
- $d_E(A, C) = 1.0$
- Theo Euclidean: Cặp gần nhất là (A, B) (vì $0.8485 < 1.0$).

• Khoảng cách Manhattan:

- $d_M(A, B) = |0.6| + |0.6| = 1.2$
- $d_M(A, C) = |1.0| + |0| = 1.0$
- Theo Manhattan: Cặp gần nhất lại là (A, C) (vì $1.0 < 1.2$)!

Như vậy, đổi metric làm thay đổi cặp điểm gần nhất.

Bài 3.3.5: Khoảng cách Hamming giữa các chuỗi ký tự

> **Đề bài:** Khoảng cách Hamming giữa hai chuỗi có cùng độ dài là số vị trí mà tại đó các ký tự tương ứng khác nhau.
>
> a. Khoảng cách Hamming có thỏa mãn 3 tiên đề của hàm khoảng cách hay không?
>
> b. Độ phức tạp của thuật toán vét cạn tìm cặp chuỗi gần nhất trong tập n chuỗi, mỗi chuỗi độ dài m, là bao nhiêu?

Phân tích phương pháp giải:

Khoảng cách Hamming H(s1, s2) = ∑ [s1[k] != s2[k]].

Lời giải chi tiết:

a. Kiểm tra 3 tiên đề metric:

1. $H(s_1, s_2) \geq 0$, và $H(s_1, s_2) = 0 \Leftrightarrow$ mọi vị trí đều giống nhau $\Leftrightarrow s_1 = s_2$. (Thỏa mãn)

2. $H(s_1, s_2) = H(s_2, s_1)$ do tính đối xứng của quan hệ khác nhau. (Thỏa mãn)

3. Bất đẳng thức tam giác: Tại vị trí thứ k, nếu $s_1[k] \neq s_2[k]$, thì bắt buộc ít nhất một trong hai khẳng định sau phải đúng: $s_1[k] \neq s_3[k]$ hoặc $s_3[k] \neq s_2[k]$. Do đó bất đẳng thức tam giác đúng cho từng vị trí và đúng cho tổng: $H(s_1, s_2) \leq H(s_1, s_3) + H(s_3, s_2)$. (Thỏa mãn)

b. Độ phức tạp thuật toán vét cạn:

Có $\frac{n(n-1)}{2}$ cặp chuỗi. Việc so sánh khoảng cách Hamming giữa hai chuỗi độ dài m tốn đúng m phép so sánh ký tự.

Tổng thời gian là: $T(n, m) = m \cdot \frac{n(n-1)}{2} \in \Theta(m \cdot n^2)$

Bài 3.3.6: Trận ném bánh kem số lẻ (Odd Pie Fight)

> **Đề bài:** Có n người (n là số lẻ, n ≥ 3) đứng trên một cánh đồng phẳng sao cho khoảng cách giữa các cặp người đôi một khác nhau (mỗi người có đúng một người láng giềng gần mình nhất). Mỗi người cầm 1 chiếc bánh kem và ném vào người gần mình nhất. Chứng minh rằng luôn luôn có ít nhất một người không bị ném trúng bánh kem.

Phân tích phương pháp giải:

Chứng minh bằng quy nạp toán học theo số lẻ n = 2k + 1.

Lời giải chi tiết:

Chứng minh bằng Quy nạp toán học:

• Cơ sở quy nạp ($n = 3$):

Gọi 3 người là A, B, C. Giả sử cặp có khoảng cách nhỏ nhất trong cả 3 người là (A, B). Khi đó, người gần A nhất chính là B, và người gần B nhất chính là A. Do đó A ném vào B và B ném vào A. Người

thứ ba C bắt buộc phải ném vào người gần C hơn giữa A và B (giả sử A). Như vậy B nhận bánh từ A, A nhận bánh từ B và C. **Người C không bị ai ném trúng!** Mệnh đề đúng với $n = 3$.

• **Bước quy nạp:** Giả sử mệnh đề đúng với mọi số lẻ $n = 2k - 1$. Ta chứng minh mệnh đề đúng với $n = 2k + 1$.

Trong n người, luôn tồn tại một cặp hai người có khoảng cách Euclidean nhỏ nhất tuyệt đối, gọi là X và Y. Vì khoảng cách đôi một khác nhau, X là người gần Y nhất và Y là người gần X nhất. Do đó X và Y sẽ ném bánh vào nhau.

Xét hai trường hợp:

1. *Trường hợp 1:* Có ít nhất một người thứ ba Z ném bánh vào X hoặc Y. Khi đó, 2 người X và Y đã nhận tổng cộng ít nhất 3 chiếc bánh. Số bánh kem còn lại để ném vào $n - 2$ người kia tối đa chỉ là $n - 3$ chiếc. Theo nguyên lý Dirichlet (Pigeonhole), $n - 2$ người mà chỉ nhận tối đa $n - 3$ chiếc bánh thì bắt buộc phải có ít nhất một người không bị ném trúng!

2. *Trường hợp 2:* Không có ai trong $n - 2$ người còn lại ném bánh vào X hoặc Y. Khi đó, X và Y ném lẫn nhau và tách rời khỏi cuộc chơi. Nhóm $n - 2$ người còn lại (là một số lẻ) chỉ ném bánh cho nhau. Theo giả thiết quy nạp, trong nhóm $n - 2$ người này chắc chắn có ít nhất 1 người không bị trúng bánh.

Mệnh đề được chứng minh hoàn toàn cho mọi số lẻ n!

Bài 3.3.7: Cặp điểm gần nhất trong không gian k chiều

> **Đề bài:** Bài toán cặp điểm gần nhất được mở rộng trong không gian k chiều với khoảng cách Euclidean giữa hai điểm p'(x'_1, ..., x'_k) và p"(x"_1, ..., x"_k) là căn bậc hai của tổng bình phương hiệu các tọa độ. Độ phức tạp của thuật toán vét cạn trong không gian k chiều là bao nhiêu?

Phân tích phương pháp giải:

Số cặp điểm vẫn là n(n-1)/2. Mỗi cặp điểm cần tính khoảng cách trên k tọa độ.

Lời giải chi tiết:

Số cặp điểm được xét trong không gian k chiều là: $\binom{n}{2} = \frac{n(n-1)}{2}$

Với mỗi cặp điểm, việc tính tổng bình phương khoảng cách $\sum_{s=1}^{k}(x'_s - x''_s)^2$ đòi hỏi:

• k phép trừ tọa độ,

• k phép nhân bình phương,

• $k - 1$ phép cộng dồn.

Tổng số phép toán số học thực hiện là: $\frac{n(n-1)}{2} \times (3k - 1) \in \Theta(k \cdot n^2)$

Với số chiều k là hằng số cố định, độ phức tạp là $\Theta(n^2)$.

Bài 3.3.8: Xác định Bao lồi và Điểm cực biên của các hình hình học

> **Đề bài:** Tìm bao lồi (Convex Hull) và xác định các điểm cực biên (Extreme Points) của các tập hợp sau:
> a. Một đoạn thẳng.
> b. Một hình vuông (miền đặc bên trong).
> c. Biên của một hình vuông (chu vi rỗng bên trong).
> d. Một đường thẳng vô hạn.

Phân tích phương pháp giải:

Điểm cực biên là điểm thuộc bao lồi mà không thể biểu diễn thành trung điểm của bất kỳ đoạn thẳng nào nối hai điểm thuộc tập hợp.

Lời giải chi tiết:

• **a. Một đoạn thẳng AB:**

- Bao lồi chính là chính đoạn thẳng AB.
- Các điểm cực biên: Đúng **2 điểm mút A và B**.

• **b. Một hình vuông đặc (cả miền trong và biên):**

- Bao lồi là chính hình vuông đó.
- Các điểm cực biên: Đúng **4 đỉnh của hình vuông**.

• **c. Biên của một hình vuông (chỉ gồm 4 cạnh, rỗng bên trong):**

- Bao lồi là toàn bộ hình vuông đặc (bao gồm cả miền trong được lấp đầy).
- Các điểm cực biên: Đúng **4 đỉnh của hình vuông**.

• **d. Một đường thẳng vô hạn:**

- Bao lồi là chính đường thẳng vô hạn đó.
- Các điểm cực biên: **KHÔNG CÓ ĐIỂM CỰC BIÊN NÀO** (vì mọi điểm trên đường thẳng vô hạn đều là điểm nằm giữa của hai điểm khác trên đường thẳng).

Bài 3.3.9: Tìm hai điểm cực biên của Bao lồi trong thời gian tuyến tính O(n)

> **Đề bài:** Thiết kế thuật toán thời gian tuyến tính $O(n)$ để xác định chắc chắn 2 điểm cực biên của bao lồi cho tập $n > 1$ điểm trong mặt phẳng.

Phân tích phương pháp giải:

Điểm có tọa độ hoành độ x nhỏ nhất (cực tả) và lớn nhất (cực hữu) bắt buộc phải là điểm cực biên của bao lồi.

Lời giải chi tiết:

Thuật toán tìm điểm cực tiểu và cực đại theo tọa độ x:

1. Quét qua n điểm của tập hợp, tìm điểm P_{min} có hoành độ x nhỏ nhất: $P_{min} = \arg min_i P_i.x$ (nếu trùng x, chọn điểm có y nhỏ nhất).

2. Đồng thời tìm điểm P_{max} có hoành độ x lớn nhất: $P_{max} = \arg max_i P_i.x$ (nếu trùng x, chọn điểm có y lớn nhất).

Chứng minh:

Toàn bộ các điểm còn lại của tập hợp đều có hoành độ x nằm trong đoạn $[P_{min}.x, P_{max}.x]$. Đường thẳng đứng $x = P_{min}.x$ tựa vào tập điểm và toàn bộ tập điểm nằm về nửa mặt phẳng bên phải của nó. Do đó, P_{min} và P_{max} không thể là điểm nằm trong đoạn thẳng nối bất kỳ cặp điểm nào khác trong tập. Vì vậy P_{min} và P_{max} bắt buộc phải là hai điểm cực biên của bao lồi!

Độ phức tạp: Chỉ cần một lượt quét tuyến tính tìm min và max: Đúng $2(n-1)$ phép so sánh, thời gian $\Theta(n)$.

Bài 3.3.10: Hiệu chỉnh thuật toán Bao lồi khi có nhiều điểm thẳng hàng

> **Đề bài:** Cần thực hiện hiệu chỉnh nào trong thuật toán vét cạn bao lồi khi có nhiều hơn 2 điểm cùng nằm trên một đường thẳng?

Phân tích phương pháp giải:

Nếu có nhiều điểm thẳng hàng trên cạnh bao lồi, biểu thức ax + by - c sẽ bằng 0 cho tất cả các điểm đó.

Lời giải chi tiết:

Hiệu chỉnh thuật toán:

Trong thuật toán gốc, ta bỏ qua điểm k khi $ax_k + by_k - c = 0$. Nếu có ≥ 3 điểm thẳng hàng nằm trên đường biên của bao lồi (ví dụ P_1, P_2, P_3 cùng nằm trên đường thẳng biên), thì CHỈ CÓ HAI ĐIỂM Ở HAI ĐẦU MÚT XA NHAU NHẤT mới là điểm cực biên và tạo thành cạnh của bao lồi, điểm nằm ở giữa không phải là điểm cực biên.

Quy tắc hiệu chỉnh cụ thể:

1. Với cặp điểm (P_i, P_j), nếu tất cả các điểm khác hoặc nằm về một phía (cùng dấu), hoặc nằm TRÊN CHÍNH ĐOẠN THẲNG đó (giá trị $= 0$):

2. Ta chỉ chấp nhận đoạn (P_i, P_j) là cạnh của bao lồi nếu **không có bất kỳ điểm nào khác nằm ngoài đoạn thẳng $P_i P_j$** dọc theo đường thẳng đó. (Nói cách khác, P_i và P_j phải là hai điểm xa nhau nhất trong số các điểm thẳng hàng).

Bài 3.3.11: Cài đặt chương trình bao lồi vét cạn hoàn chỉnh

> **Đề bài:** Mô tả thuật toán xử lý trường hợp tổng quát và cấu trúc dữ liệu để cài đặt chương trình bao lồi vét cạn.

Phân tích phương pháp giải:

Chương trình cần lưu danh sách điểm, lọc các đoạn thẳng biên hợp lệ, sắp xếp các đoạn thẳng theo vòng chu vi đa giác.

Lời giải chi tiết:

Thuật toán duyệt qua mọi cặp điểm (i, j), kiểm tra nửa mặt phẳng bằng tích chéo (cross product):

$$\text{CrossProduct}(P_i, P_j, P_k) = (P_j.x - P_i.x)(P_k.y - P_i.y) - (P_j.y - P_i.y)(P_k.x - P_i.x)$$

Nếu tất cả các điểm k đều có CrossProduct ≥ 0 (hoặc tất cả ≤ 0), ta lưu đoạn thẳng (P_i, P_j).

Sau khi có tập các đoạn thẳng biên, ta nối các đầu mút kề nhau để tạo thành chu trình đa giác bao lồi khép kín.

Bài 3.3.12: Giải bài toán Quy hoạch tuyến tính 2D bằng Điểm cực biên

> **Đề bài:** Cho bài toán quy hoạch tuyến tính nhỏ sau:
> Cực đại hóa: z = 3x + 5y
> Thỏa mãn: x + y ≤ 4
> x + 3y ≤ 6
> x ≥ 0, y ≥ 0.
> a. Vẽ miền nghiệm khả thi trong mặt phẳng tọa độ.
> b. Xác định tọa độ tất cả các điểm cực biên của miền khả thi.
> c. Giải bài toán tối ưu bằng cách tính giá trị hàm mục tiêu tại các điểm cực biên.

Phân tích phương pháp giải:

Miền khả thi là đa giác lồi bị chặn tạo bởi 4 bất phương trình. Nghiệm tối ưu luôn đạt tại ít nhất một đỉnh cực biên.

Lời giải chi tiết:

a. Miền nghiệm khả thi:

Vẽ các đường thẳng biên:

• Trục tung: $x = 0$; Trục hoành: $y = 0$.

• Đường thẳng (1): $x + y = 4$ đi qua $(4, 0)$ và $(0, 4)$. Miền nghiệm nằm phía dưới.

• Đường thẳng (2): $x + 3y = 6$ đi qua $(6, 0)$ và $(0, 2)$. Miền nghiệm nằm phía dưới.

Miền khả thi là tứ giác lồi khép kín nằm trong góc phần tư thứ nhất.

b. Tọa độ các điểm cực biên:

Tìm giao điểm của các cặp đường thẳng biên:

1. Gốc tọa độ: $O(0,0)$.

2. Giao điểm của $x = 0$ và $x + 3y = 6$: Điểm $A(0,2)$.

3. Giao điểm của $y = 0$ và $x + y = 4$: Điểm $B(4,0)$.

4. Giao điểm của $x + y = 4$ và $x + 3y = 6$:

Trừ phương trình (2) cho (1): $(x + 3y) - (x + y) = 6 - 4 \Longrightarrow 2y = 2 \Longrightarrow y = 1$.

Thay $y = 1$ vào (1): $x + 1 = 4 \Longrightarrow x = 3$.

Giao điểm là $C(3,1)$.

Tập các điểm cực biên gồm 4 điểm: **{ (0, 0), (0, 2), (4, 0), (3, 1) }**.

c. Tính giá trị hàm mục tiêu $z = 3x + 5y$ tại các điểm cực biên:

• Tại $O(0,0)$: $z = 3(0) + 5(0) = 0$.

• Tại $A(0,2)$: $z = 3(0) + 5(2) = 10$.

• Tại $B(4,0)$: $z = 3(4) + 5(0) = 12$.

• Tại $C(3,1)$: $z = 3(3) + 5(1) = 9 + 5 = 14$.

Kết luận nghiệm tối ưu:

Giá trị cực đại của bài toán là **$z_{max} = 14$**, đạt được tại điểm cực biên **$x^* = 3, y^* = 1$**.

CHUYÊN ĐỀ 3.4. Tìm kiếm toàn diện (Exhaustive Search)

Lý thuyết cốt lõi & Cơ sở toán học (Mục 3.4)

Tìm kiếm toàn diện là dạng vét cạn áp dụng cho các bài toán tối ưu hóa tổ hợp: Đòi hỏi tìm kiếm một phần tử thỏa mãn tính chất đặc biệt trong một không gian phát triển theo hàm mũ hoặc giai thừa.

1. Bài toán Người du lịch (Traveling Salesman Problem - TSP):

Cho n thành phố với bảng chi phí đi lại giữa các cặp thành phố. Tìm chu trình ngắn nhất đi qua mỗi thành phố đúng một lần rồi quay về điểm xuất phát. Không gian nghiệm: Hoán vị $(n-1)!$ chu trình khả dĩ (với đồ thị vô hướng là $\frac{(n-1)!}{2}$ chu trình).

2. Bài toán Cái ba-lô (Knapsack Problem):

Cho n vật phẩm có trọng lượng w_i và giá trị v_i, sức chứa ba-lô W. Tìm tập con các vật phẩm có tổng trọng lượng $\leq W$ và tổng giá trị cực đại. Không gian nghiệm: 2^n tập con khả dĩ.

3. Bài toán Phân công công việc (Assignment Problem):

Cho n người và n việc với ma trận chi phí $C[i,j]$. Tìm phép phân công 1-1 sao cho tổng chi phí nhỏ nhất. Không gian nghiệm: $n!$ hoán vị của các công việc.

Lời giải chi tiết 100% Toàn bộ Bài tập Mục 3.4 (11 Bài)

Bài 3.4.1: Giới hạn tính toán của Bài toán Người du lịch (TSP Exhaustive Search)

> **Đề bài:** a. Giả sử mỗi chu trình có thể sinh ra trong thời gian hằng số, lớp hiệu quả của thuật toán tìm kiếm toàn diện cho bài toán TSP là gì?
>
> b. Nếu thuật toán chạy trên máy tính thực hiện được 10 tỷ phép cộng mỗi giây (10^{10} phép tính/giây), hãy ước tính số lượng thành phố tối đa n mà bài toán có thể giải quyết được trong:

> i. 1 giờ; ii. 24 giờ; iii. 1 năm; iv. 1 thế kỷ (100 năm).

Phân tích phương pháp giải:

Với đồ thị vô hướng n đỉnh, cố định đỉnh xuất phát, số chu trình Hamilton khả dĩ là (n - 1)! / 2. Mỗi chu trình có n cạnh, chi phí tính tổng là n phép cộng. Tổng số phép tính là n * (n - 1)! / 2 = n! / 2. Nếu mỗi chu trình sinh trong O(1) và tính chi phí, số chu trình là (n - 1)! / 2.

Lời giải chi tiết:

a. Lớp hiệu quả:

Cố định đỉnh xuất phát (ví dụ đỉnh a), số lượng chu trình Hamilton khả dĩ trên đồ thị đầy đủ n đỉnh là:

• Đồ thị có hướng: $(n-1)!$ chu trình.

• Đồ thị vô hướng (chu trình xuôi và ngược có chi phí bằng nhau): $\frac{(n-1)!}{2}$ chu trình

Mỗi chu trình gồm n cạnh, tốn n phép cộng để tính tổng chi phí. Tổng thời gian chạy là: $T(n) = n \cdot \frac{(n-1)!}{2} = \frac{n!}{2} \in \Theta(n!)$

b. Ước tính số thành phố tối đa trên siêu máy tính 10^{10} phép tính/giây:

Phương trình: $\frac{n!}{2} \leq 10^{10} \times t$ (giây) $\Longrightarrow n! \leq 2 \times 10^{10} \times t$.

• i. Trong 1 giờ ($t = 3.600$ giây):

- Số phép tính tối đa: $2 \times 10^{10} \times 3.600 = 7.2 \times 10^{13}$.
- Ta có: $16! \approx 2.09 \times 10^{13} < 7.2 \times 10^{13}$ và $17! \approx 3.56 \times 10^{14} > 7.2 \times 10^{13}$.
- **Số thành phố tối đa: $n = 16$ thành phố.**

• ii. Trong 24 giờ ($t = 86.400$ giây):

- Số phép tính tối đa: $2 \times 10^{10} \times 86.400 = 1.728 \times 10^{15}$.
- Ta có: $17! \approx 3.56 \times 10^{14} < 1.728 \times 10^{15}$ và $18! \approx 6.40 \times 10^{15} > 1.728 \times 10^{15}$.
- **Số thành phố tối đa: $n = 17$ thành phố.** (Thêm 23 giờ chạy liên tục chỉ tăng được đúng 1 thành phố!)

• iii. Trong 1 năm ($t = 365 \times 86.400 \approx 3.1536 \times 10^7$ giây):

- Số phép tính tối đa: $2 \times 10^{10} \times 3.1536 \times 10^7 \approx 6.31 \times 10^{17}$.
- Ta có: $19! \approx 1.22 \times 10^{17} < 6.31 \times 10^{17}$ và $20! \approx 2.43 \times 10^{18} > 6.31 \times 10^{17}$.
- **Số thành phố tối đa: $n = 19$ thành phố.**

• iv. Trong 1 thế kỷ ($t = 100 \text{ năm} \approx 3.1536 \times 10^9$ giây):

- Số phép tính tối đa: $2 \times 10^{10} \times 3.1536 \times 10^9 \approx 6.31 \times 10^{19}$.
- Ta có: $20! \approx 2.43 \times 10^{18}$ và $21! \approx 5.11 \times 10^{19} < 6.31 \times 10^{19}$, $22! \approx 1.12 \times 10^{21}$.
- **Số thành phố tối đa: $n = 21$ thành phố.**

Bình luận sư phạm: Chạy máy tính liên tục suốt 100 năm với tốc độ 10 tỷ phép tính/giây cũng chỉ giải được bài toán tối đa $n = 21$ thành phố! Đây là minh họa kinh hoàng cho sự bùng nổ giai thừa $\Theta(n!)$.

Bài 3.4.2: Thuật toán tìm kiếm toàn diện cho Chu trình Hamilton

> **Đề bài:** Phác thảo thuật toán tìm kiếm toàn diện cho bài toán Chu trình Hamilton (Hamiltonian Circuit Problem).

Phân tích phương pháp giải:

Duyệt tất cả các hoán vị của n - 1 đỉnh (cố định đỉnh bắt đầu v1). Kiểm tra xem hoán vị đó có tạo thành chu trình hợp lệ trong đồ thị không.

Lời giải chi tiết:

Thuật toán tìm kiếm toàn diện:

1. Chọn một đỉnh bất kỳ làm đỉnh xuất phát, giả sử là đỉnh 1.

2. Sinh tất cả $(n-1)!$ hoán vị của các đỉnh còn lại $\{2,3,\dots,n\}$. Mỗi hoán vị có dạng: $v_2, v_3, \dots, v_n$.

3. Với mỗi hoán vị, xét đường đi khép kín: $1 \to v_2 \to v_3 \to \cdots \to v_n \to 1$.

4. Kiểm tra xem tất cả n cạnh trên đường đi đó có thực sự tồn tại trong đồ thị hay không:

   $(1, v_2) \in E, (v_2, v_3) \in E, \dots, (v_{n-1}, v_n) \in E, (v_n, 1) \in E$.

5. Nếu tất cả các cạnh đều tồn tại, ta đã tìm thấy một chu trình Hamilton. Dừng lại hoặc lưu vào danh sách nghiệm.

6. Nếu duyệt hết $(n-1)!$ hoán vị mà không tìm thấy đường đi nào hợp lệ, kết luận đồ thị không có chu trình Hamilton.

Độ phức tạp: Thời gian kiểm tra mỗi hoán vị là $O(n)$, tổng thời gian chạy là $O(n \cdot (n-1)!) = O(n!)$.

Bài 3.4.3: Kiểm tra Chu trình Euler trên đồ thị biểu diễn bằng ma trận kề

> **Đề bài:** Phác thảo một thuật toán để xác định xem một đồ thị vô hướng liên thông biểu diễn bằng ma trận kề có chứa Chu trình Euler (Eulerian Circuit) hay không? Xác định lớp hiệu quả thời gian của thuật toán đó.

Phân tích phương pháp giải:

Định lý Euler (1736): Một đồ thị vô hướng liên thông có chu trình Euler khi và chỉ khi MỌI đỉnh của đồ thị đều có BẬC CHẴN.

Lời giải chi tiết:

Thuật toán dựa trên Định lý Euler:

Theo Định lý Euler kinh điển: Một đồ thị vô hướng liên thông chứa một chu trình Euler khi và chỉ khi **bậc của tất cả các đỉnh trong đồ thị đều là số chẵn**.

Với đồ thị biểu diễn bằng ma trận kề $A[0..n-1,0..n-1]$, bậc của đỉnh i chính bằng tổng các phần tử trên hàng i của ma trận.

Mã giả chuẩn hóa:

```text
function:  HasEulerianCircuit(A, n): boolean
  input:  A: array [0..n - 1, 0..n - 1] of boolean – ma trận kề của đồ thị liên thông.
          n: integer – số lượng đỉnh.
 output:  hasEuler: boolean – true nếu có chu trình Euler, ngược lại false.
1   for i = 0 to n - 1 do
2       degree = 0
3       for j = 0 to n - 1 do
4           if A[i, j] = true then degree = degree + 1 end-if
5       end-for
6       if degree mod 2 ≠ 0 then
7           return false – phát hiện đỉnh có bậc lẻ, không thể có chu trình Euler.
8       end-if
9   end-for
10  return true – tất cả các đỉnh đều có bậc chẵn.
```

Độ phức tạp thời gian: Thuật toán duyệt qua toàn bộ $n \times n$ phần tử của ma trận kề để tính bậc: $T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 = n^2 \in \Theta(n^2)$

(Nếu biểu diễn bằng danh sách kề, ta có thể kiểm tra trong thời gian tuyến tính $\Theta(|V| + |E|)$).

Bài 3.4.4: Hoàn thành vét cạn bài toán Phân công công việc trong giáo trình

> **Đề bài:** Hoàn thành việc áp dụng tìm kiếm toàn diện cho bài toán Phân công công việc (Assignment Problem) với ma trận chi phí 4 người và 4 việc bắt đầu ở trang 119 của giáo trình:
> C = [[9, 2, 7, 8], [6, 4, 3, 7], [5, 8, 1, 8], [7, 6, 9, 4]]. Liệt kê tất cả 24 hoán vị và xác định phép phân công tối ưu có tổng chi phí nhỏ nhất.

Phân tích phương pháp giải:

Có 4! = 24 hoán vị của 4 công việc giao cho 4 người. Tính tổng C[1, j1] + C[2, j2] + C[3, j3] + C[4, j4] cho cả 24 hoán vị.

Lời giải chi tiết:

Ma trận chi phí C:

• Người 1: Việc 1=9, Việc 2=2, Việc 3=7, Việc 4=8

• Người 2: Việc 1=6, Việc 2=4, Việc 3=3, Việc 4=7

• Người 3: Việc 1=5, Việc 2=8, Việc 3=1, Việc 4=8

• Người 4: Việc 1=7, Việc 2=6, Việc 3=9, Việc 4=4

Bảng tính chi phí toàn bộ 24 hoán vị phân công $\langle j_1, j_2, j_3, j_4 \rangle$:

1. $\langle 1,2,3,4 \rangle: 9 + 4 + 1 + 4 = 18$
2. $\langle 1,2,4,3 \rangle: 9 + 4 + 8 + 9 = 30$
3. $\langle 1,3,2,4 \rangle: 9 + 3 + 8 + 4 = 24$
4. $\langle 1,3,4,2 \rangle: 9 + 3 + 8 + 6 = 26$
5. $\langle 1,4,2,3 \rangle: 9 + 7 + 8 + 9 = 33$
6. $\langle 1,4,3,2 \rangle: 9 + 7 + 1 + 6 = 23$
7. $\langle 2,1,3,4 \rangle: 2 + 6 + 1 + 4 = 13$ <-- **(TỐI ƯU CỰC TIỂU!)**
8. $\langle 2,1,4,3 \rangle: 2 + 6 + 8 + 9 = 25$
9. $\langle 2,3,1,4 \rangle: 2 + 3 + 5 + 4 = 14$
10. $\langle 2,3,4,1 \rangle: 2 + 3 + 8 + 7 = 20$
11. $\langle 2,4,1,3 \rangle: 2 + 7 + 5 + 9 = 23$
12. $\langle 2,4,3,1 \rangle: 2 + 7 + 1 + 7 = 17$
13. $\langle 3,1,2,4 \rangle: 7 + 6 + 8 + 4 = 25$
14. $\langle 3,1,4,2 \rangle: 7 + 6 + 8 + 6 = 27$
15. $\langle 3,2,1,4 \rangle: 7 + 4 + 5 + 4 = 20$
16. $\langle 3,2,4,1 \rangle: 7 + 4 + 8 + 7 = 26$
17. $\langle 3,4,1,2 \rangle: 7 + 7 + 5 + 6 = 25$
18. $\langle 3,4,2,1 \rangle: 7 + 7 + 8 + 7 = 29$
19. $\langle 4,1,2,3 \rangle: 8 + 6 + 8 + 9 = 31$
20. $\langle 4,1,3,2 \rangle: 8 + 6 + 1 + 6 = 21$
21. $\langle 4,2,1,3 \rangle: 8 + 4 + 5 + 9 = 26$
22. $\langle 4,2,3,1 \rangle: 8 + 4 + 1 + 7 = 20$
23. $\langle 4,3,1,2 \rangle: 8 + 3 + 5 + 6 = 22$
24. $\langle 4,3,2,1 \rangle: 8 + 3 + 8 + 7 = 26$

Kết luận nghiệm tối ưu:

Phương án phân công tối ưu nhất là hoán vị **$\langle 2, 1, 3, 4 \rangle$** với tổng chi phí nhỏ nhất là **$C_{min} = 13$**:

• Người 1 làm việc 2 (chi phí = 2)

• Người 2 làm việc 1 (chi phí = 6)

• Người 3 làm việc 3 (chi phí = 1)

• Người 4 làm việc 4 (chi phí = 4)

Bài 3.4.5: Phản ví dụ thuật toán tham lam cho Phân công công việc

> **Đề bài:** Đưa ra một ví dụ về bài toán Phân công công việc có kích thước nhỏ nhất mà trong đó nghiệm tối ưu KHÔNG CHỨA phần tử có chi phí nhỏ nhất trong ma trận chi phí.

Phân tích phương pháp giải:

Tìm ma trận 2 x 2 trong đó phần tử nhỏ nhất dẫn đến phần tử còn lại quá lớn.

Lời giải chi tiết:

Xét ma trận chi phí cấp 2×2 sau:

$$C = \begin{pmatrix} 1 & 100 \\ 2 & 3 \end{pmatrix}$$

Phần tử nhỏ nhất trong toàn bộ ma trận là $C[1,1] = 1$.

Chỉ có 2 phương án phân công khả dĩ:

1. **Phương án 1 (chọn phần tử nhỏ nhất $C[1, 1] = 1$):**

   Gán Người 1 cho Việc 1 ($C[1,1] = 1$). Bắt buộc Người 2 phải làm Việc 2 ($C[2,2] = 3$).

   Tổng chi phí: $1 + 3 = 4$.

Nhưng nếu sửa ma trận một chút:

$$C = \begin{pmatrix} 1 & 100 \\ 100 & 2 \end{pmatrix}$$

Ở ma trận này:

• Phương án 1 (chọn $C[1,1] = 1$): Người 2 buộc phải làm việc 2 với giá 2 -> tổng là $1 + 2 = 3$ (vẫn chứa 1).

Để KHÔNG chứa phần tử nhỏ nhất, ta thiết lập ma trận:

$$C = \begin{pmatrix} 1 & 10 \\ 1 & 2 \end{pmatrix}$$

Hoặc chuẩn nhất:

$$C = \begin{pmatrix} 1 & 5 \\ 2 & 100 \end{pmatrix}$$

• Phương án $\langle 1,2 \rangle$: $C[1,1] + C[2,2] = 1 + 100 = 101$ (chứa phần tử nhỏ nhất 1).

• Phương án $\langle 2,1 \rangle$: $C[1,2] + C[2,1] = 5 + 2 = 7$.

Rõ ràng nghiệm tối ưu là phương án $\langle 2,1 \rangle$ với chi phí bằng **7**, và nó **HOÀN TOÀN KHÔNG CHỨA phần tử nhỏ nhất $C[1, 1] = 1$**!

Bài 3.4.6: Bài toán Phân hoạch tập hợp (Partition Problem)

> **Đề bài:** Cho n số nguyên dương. Phân hoạch chúng thành hai tập con rời rạc có tổng các phần tử bằng nhau. Thiết kế thuật toán tìm kiếm toàn diện sao cho số tập con cần sinh là nhỏ nhất.

Phân tích phương pháp giải:

Tính tổng S của toàn bộ n số. Nếu S lẻ, lập tức kết luận không thể chia đều. Nếu S chẵn, mục tiêu là tìm một tập con có tổng đúng bằng S/2. Vì tập A và tập bù B đối xứng, ta chỉ cần sinh 2^(n-1) tập con.

Lời giải chi tiết:

Thuật toán tối thiểu hóa số tập con sinh:

1. Tính tổng tất cả các phần tử $S = \sum_{i=1}^{n} A[i]$.

2. Nếu S là số lẻ ($Smod2 \neq 0$): Dừng ngay lập tức và trả về false (vì hai số nguyên bằng nhau không thể có tổng là số lẻ).

3. Nếu S chẵn: Bài toán quy về tìm một tập con $A' \subset A$ có tổng đúng bằng $S/2$.

4. Để tránh sinh trùng lặp đối xứng (tập con A' và tập phần bù $A - A'$ có vai trò như nhau), ta **cố định phần tử đầu tiên $A[1]$ luôn nằm trong tập A'**.

5. Duyệt qua tất cả 2^{n-1} tập con khả dĩ được tạo bởi $n-1$ phần tử còn lại $\{A[2], \dots, A[n]\}$, tính tổng các phần tử cộng thêm $A[1]$:

   • Nếu tổng $= S/2$: Tìm thấy phân hoạch thành công! Trả về hai tập con.

6. Nếu duyệt hết 2^{n-1} tập con mà không có tập nào đạt tổng $S/2$, trả về false.

Độ phức tạp: Số tập con cần sinh giảm một nửa: từ 2^n xuống đúng $2^{n-1} \in \Theta(2^n)$.

Bài 3.4.7: Bài toán Tìm đồ thị con đầy đủ (Clique Problem)

> **Đề bài:** Cho đồ thị G và số nguyên dương k. Xác định xem G có chứa một clique kích thước k (đồ thị con đầy đủ gồm k đỉnh) hay không. Thiết kế thuật toán tìm kiếm toàn diện cho bài toán này.

Phân tích phương pháp giải:

Một clique kích thước k là tập k đỉnh mà giữa hai đỉnh bất kỳ đều có cạnh nối. Sinh tất cả các tập con k đỉnh từ n đỉnh: có C(n, k) tập con.

Lời giải chi tiết:

Mã giả chuẩn hóa:

```text
function:  HasCliqueK(G, n, k): boolean
  input:  G = (V, E): graph; n: integer; k: integer (1 ≤ k ≤ n).
 output:  hasClique: boolean – true nếu có clique kích thước k.
```

```
1  for each tập con S ⊆ V gồm đúng k đỉnh do – có C(n, k) tập con
2      is_clique = true
3      for each cặp đỉnh u, v ∈ S với u ≠ v do – có k(k-1)/2 cặp
4          if not IsEdge(G, u, v) then
5              is_clique = false
6              break
7          end-if
8      end-for each
9      if is_clique then return true end-if
10 end-for each
11 return false.
```

Độ phức tạp thời gian:

Số tập con kích thước k là $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Việc kiểm tra mỗi tập con tốn $\binom{k}{2} = \frac{k(k-1)}{2}$ phép kiểm tra cạnh.

Tổng thời gian: $T(n,k) \in O\left(k^2\binom{n}{k}\right) = O\left(k^2 \frac{n^k}{k!}\right)$

Bài 3.4.8: Ứng dụng Tìm kiếm toàn diện cho Bài toán Sắp xếp (Permutation Sort / BogoSort)

> **Đề bài:** Giải thích cách áp dụng tìm kiếm toàn diện cho bài toán sắp xếp mảng và xác định lớp hiệu quả của thuật toán đó.

Phân tích phương pháp giải:

Không gian nghiệm của việc sắp xếp mảng n phần tử là n! hoán vị. Thuật toán sinh từng hoán vị và kiểm tra xem nó đã có thứ tự tăng dần chưa.

Lời giải chi tiết:

Thuật toán Sắp xếp bằng Tìm kiếm toàn diện (Permutation Sort / Bogosort):

1. Sinh lần lượt tất cả $n!$ hoán vị khả dĩ của mảng đầu vào $A[0..n-1]$.

2. Với mỗi hoán vị P, quét từ đầu đến cuối kiểm tra xem các phần tử có thỏa mãn thứ tự không giảm hay không: $P[i] \leq P[i+1]$ với mọi $0 \leq i \leq n-2$ (tốn $n-1$ phép so sánh).

3. Ngay khi tìm thấy một hoán vị có thứ tự tăng dần, trả về hoán vị đó và dừng thuật toán.

Lớp hiệu quả thời gian:

• Trường hợp tốt nhất: Hoán vị đầu tiên đã có thứ tự sẵn: $O(n)$.

• Trường hợp xấu nhất: Hoán vị duy nhất có thứ tự nằm ở cuối cùng của danh sách $n!$ hoán vị: $C_{worst}(n) = (n-1) \cdot n! \in \Theta(n \cdot n!)$

Đây là một trong những thuật toán sắp xếp tồi tệ và phi hiệu quả nhất trong lịch sử khoa học máy tính.

Bài 3.4.9: Bài toán Tám con hậu (Eight-Queens Exhaustive Search)

> **Đề bài:** Xét bài toán kinh điển đặt 8 con hậu lên bàn cờ 8 × 8 sao cho không có 2 con nào ăn nhau. Có bao nhiêu cấu hình khả dĩ nếu:
>
> a. Không có 2 con hậu nào nằm trên cùng một ô cờ?
>
> b. Không có 2 con hậu nào nằm trên cùng một hàng?
>
> c. Không có 2 con hậu nào nằm trên cùng một hàng hoặc cùng một cột?
>
> Ước tính thời gian duyệt hết các cấu hình trên máy tính kiểm tra được 10 tỷ vị trí/giây (10^{10} pos/sec).

Phân tích phương pháp giải:

a. Chọn 8 ô từ 64 ô: C(64, 8).

b. Mỗi hàng chọn 1 ô từ 8 ô: 8^8.

c. Mỗi hàng 1 ô và các cột khác nhau: 8! hoán vị.

Lời giải chi tiết:

a. Không có 2 hậu nào trên cùng 1 ô (Chọn 8 ô tùy ý từ 64 ô cờ):

Số cấu hình là tổ hợp chập 8 của 64:

$$N_a = \binom{64}{8} = \frac{64!}{8! \cdot 56!} = 4.426.165.368 \approx 4.43 \times 10^9 \text{ cấu hình}$$

• Thời gian trên máy tính 10^{10} pos/sec: $t_a = \frac{4.43 \times 10^9}{10^{10}} \approx 0.44$ giây.

b. Không có 2 hậu nào trên cùng 1 hàng (Mỗi hàng đúng 1 hậu):

Hàng 1 có 8 cách chọn cột, hàng 2 có 8 cách, ..., hàng 8 có 8 cách:

$$N_b = 8^8 = 16.777.216 \approx 1.68 \times 10^7 \text{ cấu hình}$$

• Thời gian trên máy tính: $t_b = \frac{1.68 \times 10^7}{10^{10}} \approx 0.00168$ giây $= 1.68$ mili-giây.

c. Không có 2 hậu nào cùng hàng hoặc cùng cột (Hoán vị 8 cột):

Hàng 1 có 8 cách chọn, hàng 2 còn 7 cách, ..., hàng 8 còn 1 cách:

$$N_c = 8! = 40.320 \text{ cấu hình}$$

• Thời gian trên máy tính: $t_c = \frac{40.320}{10^{10}} \approx 4 \times 10^{-6}$ giây $= 4$ micro-giây!

Nhận xét: Việc áp dụng ràng buộc hình học (mỗi hàng 1 hậu, mỗi cột 1 hậu) đã làm giảm không gian tìm kiếm từ 4.4 tỷ xuống còn vỏn vẹn 40.320 trạng thái!

Bài 3.4.10: Ma phương bậc n (Magic Squares)

> **Đề bài:** Ma phương bậc n là bảng n × n chứa các số từ 1 đến n² sao cho tổng mỗi hàng, mỗi cột và 2 đường chéo chính đều bằng nhau.
> a. Chứng minh rằng nếu ma phương bậc n tồn tại, tổng ma thuật bắt buộc phải bằng s = n(n² + 1) / 2.
> b. Thiết kế thuật toán tìm kiếm toàn diện sinh tất cả ma phương bậc n.
> c. Nêu thuật toán hiệu quả hơn để sinh ma phương bậc lẻ (Thuật toán Xiêm - Siamese Method).

Phân tích phương pháp giải:

a. Tổng tất cả các số từ 1 đến n² chia đều cho n hàng.

b. Sinh tất cả (n²)! hoán vị của các số từ 1 đến n².

c. Thuật toán De la Loubère (Siamese method) đi chéo lên trên sang phải.

Lời giải chi tiết:

a. Chứng minh tổng ma thuật $s = \frac{n(n^2 + 1)}{2}$:

Tổng của tất cả các số nguyên từ 1 đến n^2 là:

$$S_{total} = \sum_{i=1}^{n^2} i = \frac{n^2(n^2+1)}{2}$$

Vì ma phương có n hàng và mỗi hàng đều có tổng bằng s, nên tổng của cả n hàng chính bằng S_{total}:

$$n \cdot s = S_{total} = \frac{n^2(n^2+1)}{2} \Rightarrow s = \frac{n(n^2+1)}{2}$$

(Ví dụ với $n = 3$: $s = \frac{3(3^2+1)}{2} = \frac{3\times10}{2} = 15$).

b. Thuật toán tìm kiếm toàn diện:

1. Sinh tất cả $(n^2)!$ hoán vị của tập hợp các số $\{1,2,\dots,n^2\}$.

2. Với mỗi hoán vị, điền vào bảng $n \times n$.

3. Kiểm tra xem tổng của n hàng, n cột và 2 đường chéo có cùng bằng $s = \frac{n(n^2+1)}{2}$ hay không.

Độ phức tạp cực lớn: $O(n^2 \cdot (n^2)!)$. Với $n = 3$, $(9)! = 362.880$ hoán vị; với $n = 4$, $(16)! \approx 2.09 \times 10^{13}$.

c. Thuật toán Siamese (De la Loubère) cho n lẻ:

Đặt số 1 ở ô giữa của hàng đầu tiên. Quy tắc: Đi theo đường chéo lên trên sang phải (nếu vượt biên thì cuộn vòng quanh bảng). Nếu ô đã có số, đi lùi xuống 1 ô ngay dưới số vừa điền. Thuật toán chạy trong thời gian tuyến tính $\Theta(n^2)$!

Bài 3.4.11: Bài toán Giải đố chữ số kinh điển: SEND + MORE = MONEY

> **Đề bài:** Giải bài toán đố chữ số nổi tiếng của Henry Dudeney (1924):
> S E N D

> + M O R E
> ---
> M O N E Y
> Mỗi chữ cái đại diện cho 1 chữ số duy nhất từ 0 đến 9, các chữ cái khác nhau nhận chữ số khác nhau, và chữ số đầu tiên (S, M) khác 0.
> a. Mô tả thuật toán tìm kiếm toàn diện.
> b. Trình bày lời giải suy luận logic toán học chuẩn mực.

Phân tích phương pháp giải:

Có 8 chữ cái: S, E, N, D, M, O, R, Y. Chọn 8 chữ số phân biệt từ 10 chữ số: P(10, 8) = 1.814.400 hoán vị.

Lời giải chi tiết:

a. Thuật toán tìm kiếm toàn diện:

Duyệt qua tất cả $P(10,8) = \frac{10!}{2!} = 1.814.400$ phép gán chữ số cho 8 chữ cái (S, E, N, D, M, O, R, Y). Bỏ qua các trường hợp có $S = 0$ hoặc $M = 0$. Kiểm tra xem đẳng thức số học sau có thỏa mãn hay không:

$$(1000S + 100E + 10N + D) + (1000M + 100O + 10R + E) = \\ = 10000M + 1000O + 100N + 10E + Y$$

b. Suy luận logic toán học (Lời giải của Dudeney, 1924):

1. **Xác định M:** Phép cộng hai số 4 chữ số tạo ra số 5 chữ số $\Longrightarrow$ số nhớ ở cột cao nhất chỉ có thể là 1 $\Longrightarrow$ **M = 1**.

2. **Xác định O và S:** Ở cột nghìn: $S + M +$ nhớ $= S + 1 +$ nhớ ≥ 10. Vì $M = 1$, tổng này tạo ra $10 + O$. Do $S \leq 9$, $S + 1 +$ nhớ $\leq 9 + 1 + 1 = 11$. Vì $M = 1$ nên O không thể bằng 1 $\Longrightarrow$ **O = 0**. Để có số nhớ sang hàng vạn với $O = 0$, ta buộc phải có $S + 1 = 10$ hoặc $S + 1 + 1 = 10 \Longrightarrow$ **S = 9** (nếu có nhớ từ cột trăm) hoặc $S = 8$ (nếu có nhớ 1 từ cột trăm và $S + 1 + 1 = 10$, nhưng điều này đòi hỏi $E + 0 +$ nhớ $\geq 10 \Longrightarrow E = 9$, trùng với S, loại!). Vậy **S = 9** và không có nhớ sang cột chục nghìn.

3. **Xác định E và N:** Cột trăm: $E + O +$ nhớ $= N \Longrightarrow E + 0 +$ nhớ $= N$. Vì $E \neq N$, bắt buộc phải có nhớ 1 từ cột chục chuyển sang $\Longrightarrow$ **N = E + 1**.

4. **Xác định R:** Cột chục: $N + R +$ nhớ $= E + 10$ (vì có nhớ 1 sang cột trăm). Thay $N = E + 1$ vào: $(E + 1) + R +$ nhớ $= E + 10 \Longrightarrow R + 1 +$ nhớ $= 10 \Longrightarrow R +$ nhớ $= 9$. Vì $S = 9$ nên R không thể bằng $9 \Longrightarrow$ phải có nhớ 1 từ cột đơn vị $\Longrightarrow$ **R = 8** và cột đơn vị có nhớ 1.

5. **Xác định D và Y:** Cột đơn vị: $D + E = Y + 10$ (có nhớ 1). Các chữ số còn lại chưa dùng là $\{2, 3, 4, 5, 6, 7\}$. Vì $N = E + 1$, ta cần hai số liên tiếp trong tập còn lại. Thử $E = 5 \Longrightarrow N = 6$. Khi đó $D + 5 = Y + 10 \Longrightarrow D - Y = 5$. Trong các số còn lại $\{2, 3, 4, 7\}$, cặp duy nhất có hiệu bằng 5 là $7 - 2 = 5 \Longrightarrow$ **D = 7** và **Y = 2**.

Đáp án duy nhất:

```text
```

```
  S E N D      9 5 6 7
+ M O R E    + 1 0 8 5
 ---------    ----------
 M O N E Y    1 0 6 5 2
```

Tương ứng: **S = 9, E = 5, N = 6, D = 7, M = 1, O = 0, R = 8, Y = 2**.

CHUYÊN ĐỀ 3.5. Duyệt đồ thị theo chiều sâu (DFS) và Chiều rộng (BFS)

Lý thuyết cốt lõi & Cơ sở toán học (Mục 3.5)

Duyệt đồ thị là kỹ thuật duyệt qua toàn bộ các đỉnh và cạnh của đồ thị một cách có hệ thống.

1. Duyệt theo chiều sâu (Depth-First Search - DFS):

Bắt đầu từ một đỉnh, đi sâu nhất có thể dọc theo các cạnh chưa thăm. Khi gặp 'đường cụt' (dead end - đỉnh không còn láng giềng chưa thăm), thuật toán lùi (backtrack) về đỉnh trước đó. Sử dụng cấu trúc **Ngăn xếp (Stack)** hoặc đệ quy. Cạnh đồ thị được phân loại thành: Cạnh cây (Tree edges) và Cạnh ngược (Back edges - nối về tổ tiên, chỉ báo chu trình).

2. Duyệt theo chiều rộng (Breadth-First Search - BFS):

Khám phá đồ thị theo từng tầng khoảng cách: Thăm đỉnh xuất phát, sau đó thăm tất cả các láng giềng kề nó, rồi tới các láng giềng cách 2 cạnh, v.v. Sử dụng cấu trúc **Hàng đợi (Queue - FIFO)**. Cạnh đồ thị được phân thành: Cạnh cây (Tree edges) và Cạnh chéo (Cross edges - nối giữa các đỉnh cùng tầng hoặc hai tầng kề nhau).

Các thuật toán kinh điển chuẩn hóa (Mục 3.5)

Thuật toán ALGO_3.7: DFS(G)

Bài toán: Duyệt toàn bộ đỉnh và cạnh của đồ thị G bằng chiến lược đi sâu nhất.

Ý tưởng thiết kế: Duyệt đệ quy từ đỉnh v, đánh dấu visited, ghi nhận thứ tự duyệt và phân loại cạnh.

```
function: DFS(G): void
   input: G = (V, E): graph  – đồ thị biểu diễn bằng danh sách kề hoặc ma trận kề.
  global: visited: array [1..|V|] of boolean  – mảng đánh dấu đỉnh đã thăm.
          count: integer  – bộ đếm thời gian duyệt.
  output: Rừng DFS và thứ tự thăm các đỉnh.

 1  count = 0
 2  for each đỉnh v ∈ V do
 3      visited[v] = false
 4  end-for each
```

```
 5  for each đỉnh v ∈ V do
 6      if not visited[v] then
 7          dfs(v)  - khởi động một cây DFS mới trong rừng.
 8      end-if
 9  end-for each
10  return.
```

Phân tích toán học: Với danh sách kề: Mỗi đỉnh được gọi dfs đúng 1 lần, mỗi cạnh (u, v) được xét đúng 2 lần (đồ thị vô hướng) hoặc 1 lần (có hướng): $T(|V|, |E|) \in \Theta(|V| + |E|)$

Với ma trận kề: Tại mỗi đỉnh, phải quét toàn bộ dòng kích thước $|V|$ để tìm láng giềng: $T(|V|) \in \Theta(|V|^2)$

Độ phức tạp tính toán: {'basic_op': 'Phép kiểm tra đỉnh đã thăm và duyệt cạnh kề.', 'best': '$Theta(|V| + |E|)$ với danh sách kề.', 'worst': '$Theta(|V| + |E|)$ với danh sách kề ($Theta(|V|^2)$ với ma trận kề).', 'avg': '$Theta(|V| + |E|)$.', 'space': '$Theta(|V|)$ cho mảng visited và ngăn xếp đệ quy.'}

Nhận xét & Tối ưu hóa: Ứng dụng trọng tâm của DFS: Kiểm tra tính liên thông, phát hiện chu trình (tồn tại cạnh ngược back edge), sắp xếp tô-pô, tìm khớp và cầu.

Thuật toán ALGO_3.8: BFS(G)

Bài toán: Duyệt đồ thị theo từng tầng khoảng cách từ đỉnh nguồn bằng hàng đợi FIFO.

Ý tưởng thiết kế: Đưa đỉnh nguồn vào hàng đợi Q. Lặp lại việc lấy đỉnh v ra và đưa tất cả các láng giềng chưa thăm của v vào cuối hàng đợi.

```
function: BFS(G): void
   input: G = (V, E): graph  - đồ thị biểu diễn bằng danh sách kề.
  global: visited: array [1..|V|] of boolean  - mảng đánh dấu.
          count: integer  - thứ tự thăm.
  output: Rừng BFS và đường đi ngắn nhất (số cạnh) từ nguồn tới các đỉnh.

 1  count = 0
 2  for each đỉnh v ∈ V do
 3      visited[v] = false
 4  end-for each
 5  Q = EmptyQueue()
 6  for each đỉnh v ∈ V do
 7      if not visited[v] then
 8          visited[v] = true
 9          Enqueue(Q, v)
10          while not IsEmpty(Q) do
11              u = Dequeue(Q)
12              count = count + 1
13              for each đỉnh w kề với u do
14                  if not visited[w] then
```

```
                    visited[w] = true
                    Enqueue(Q, w)
                end-if
            end-for each
        end-while
    end-if
end-for each
return.
```

Phân tích toán học: Mỗi đỉnh vào và ra hàng đợi đúng một lần. Mọi cạnh kề được duyệt qua đúng một lần từ mỗi đầu mút: $T(|V|, |E|) \in \Theta(|V| + |E|)$

Với ma trận kề: Chi phí là $\Theta(|V|^2)$.

Độ phức tạp tính toán: {'basic_op': 'Thao tác Enqueue/Dequeue và duyệt cạnh kề.', 'best': '$Theta(|V| + |E|)$ với danh sách kề.', 'worst': '$Theta(|V| + |E|)$.', 'avg': '$Theta(|V| + |E|)$.', 'space': '$Theta(|V|)$ cho mảng visited và hàng đợi Q.'}

Nhận xét & Tối ưu hóa: BFS luôn tìm ra đường đi ngắn nhất (ít cạnh nhất) từ đỉnh nguồn đến mọi đỉnh khác trên đồ thị không có trọng số.

Lời giải chi tiết 100% Toàn bộ Bài tập Mục 3.5 (11 Bài)

Bài 3.5.1: Biểu diễn đồ thị và duyệt DFS (Adjacency Matrix, Adjacency Lists, DFS Tree Trace)

Đề bài: Cho đồ thị vô hướng gồm 7 đỉnh a, b, c, d, e, f, g với các cạnh:

$(a, b), (a, c), (a, d), (a, e), (b, d), (b, f), (c, g), (d, f), (e, g)$.

a. Viết ma trận kề (adjacency matrix) và danh sách kề (adjacency lists) biểu diễn đồ thị này. (Giả sử các hàng, cột của ma trận và các đỉnh trong danh sách kề được sắp theo thứ tự bảng chữ cái).

b. Bắt đầu từ đỉnh a và ưu tiên chọn đỉnh kề theo thứ tự bảng chữ cái khi có nhiều lựa chọn, hãy duyệt đồ thị bằng tìm kiếm theo chiều sâu (DFS) và dựng cây DFS tương ứng. Chỉ ra thứ tự các đỉnh được duyệt lần đầu tiên (được đẩy vào ngăn xếp - push) và thứ tự các đỉnh trở thành ngõ cụt (được lấy ra khỏi ngăn xếp - pop).

Gợi ý của tác giả (Levitin): a. Áp dụng định nghĩa ma trận kề và danh sách kề từ Mục 1.4. b. Thực hiện duyệt DFS từng bước tương tự Hình 3.10 trong sách giáo trình.

Phân tích phương pháp giải:

Đồ thị có $|V| = 7$ đỉnh và $|E| = 9$ cạnh. Bậc của các đỉnh:

- $\deg(a) = 4$: kề với b, c, d, e

- $\deg(b) = 3$: kề với a, d, f

- $\deg(c) = 2$: kề với a, g

- $\deg(d) = 3$: kề với a, b, f

- $\deg(e) = 2$: kề với a, g

- $\deg(f) = 2$: kề với b, d

- $\deg(g) = 2$: kề với c, e

Tổng bậc: $4 + 3 + 2 + 3 + 2 + 2 + 2 = 18 = 2 \times 9$.

Khi duyệt DFS từ a:

1. Ngăn xếp lưu đường đi hiện tại từ gốc đến đỉnh đang xét.

2. Tại mỗi đỉnh, chọn đỉnh kề chưa thăm có nhãn chữ cái nhỏ nhất.

3. Khi gặp ngõ cụt (tất cả đỉnh kề đều đã được thăm), đỉnh đó được pop khỏi ngăn xếp và quay lui (backtrack).

Lời giải chi tiết:

a. Biểu diễn đồ thị:

1. Ma trận kề (Adjacency Matrix A kích thước 7×7):

Các hàng và cột theo thứ tự a, b, c, d, e, f, g:

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	0	1	0
c	1	0	0	0	0	0	1
d	1	1	0	0	0	1	0
e	1	0	0	0	0	0	1
f	0	1	0	1	0	0	0
g	0	0	1	0	1	0	0

2. Danh sách kề (Adjacency Lists):

- $a \to b \to c \to d \to e$

- $b \to a \to d \to f$

- $c \to a \to g$

- $d \to a \to b \to f$

- $e \to a \to g$

- $f \to b \to d$

- $g \to c \to e$

b. Quá trình duyệt DFS từ đỉnh a (ưu tiên chữ cái):

1. **Thăm đỉnh a:** Đẩy a vào ngăn xếp. Ngăn xếp: $[a]$. Đỉnh kề chưa thăm nhỏ nhất là b.

2. **Thăm đỉnh b (từ a):** Cạnh cây (a, b). Đẩy b vào ngăn xếp: $[a, b]$. Đỉnh kề chưa thăm nhỏ nhất của b là d.

3. **Thăm đỉnh d (từ b):** Cạnh cây (b, d). Đẩy d vào ngăn xếp: $[a, b, d]$. Đỉnh kề chưa thăm nhỏ nhất của d là f (vì a, b đã thăm; cạnh (d, a) là cạnh ngược - back edge).

4. **Thăm đỉnh f (từ d):** Cạnh cây (d, f). Đẩy f vào ngăn xếp: $[a, b, d, f]$.

- Xét đỉnh kề của f: b (đã thăm $\Longrightarrow$ cạnh ngược (f, b)), d (cha, đã thăm).

- f không còn đỉnh kề chưa thăm $\Longrightarrow$ f trở thành ngõ cụt đầu tiên. **POP f**.

5. **Quay lui về d:** Đỉnh d đã xét hết đỉnh kề $\Longrightarrow$ d là ngõ cụt tiếp theo. **POP d**.

6. **Quay lui về b:** Đỉnh b đã xét hết đỉnh kề $\Longrightarrow$ b là ngõ cụt tiếp theo. **POP b**.

7. **Quay lui về a:** Ngăn xếp còn $[a]$. Các đỉnh kề chưa thăm của a theo thứ tự là c, e (đỉnh d đã thăm ở bước 3).

8. **Thăm đỉnh c (từ a):** Cạnh cây (a, c). Đẩy c vào ngăn xếp: $[a, c]$. Đỉnh kề chưa thăm của c là g.

9. **Thăm đỉnh g (từ c):** Cạnh cây (c, g). Đẩy g vào ngăn xếp: $[a, c, g]$. Đỉnh kề chưa thăm của g là e.

10. **Thăm đỉnh e (từ g):** Cạnh cây (g, e). Đẩy e vào ngăn xếp: $[a, c, g, e]$.

- Xét đỉnh kề của e: a (đã thăm $\Longrightarrow$ cạnh ngược (e, a)), g (cha, đã thăm).

- e không còn đỉnh kề chưa thăm $\Longrightarrow$ e là ngõ cụt. **POP e**.

11. **Quay lui về g:** Hết đỉnh kề chưa thăm $\Longrightarrow$ **POP g**.

12. **Quay lui về c:** Hết đỉnh kề chưa thăm $\Longrightarrow$ **POP c**.

13. **Quay lui về a:** Hết đỉnh kề chưa thăm $\Longrightarrow$ **POP a**.

Tổng kết kết quả:

- Thứ tự các đỉnh được chạm đến lần đầu (Push order):

a, b, d, f, c, g, e

- Thứ tự các đỉnh trở thành ngõ cụt (Pop order):

f, d, b, e, g, c, a

- Tập cạnh cây (Tree edges, 6 cạnh):

$\{(a, b), (b, d), (d, f), (a, c), (c, g), (g, e)\}$

- Tập cạnh ngược (Back edges, 3 cạnh):

$\{(d, a), (f, b), (e, a)\}$

Tổng số cạnh = $6 + 3 = 9$ cạnh, đúng bằng $|E|$.

Bài 3.5.2: Hiệu quả của DFS trên đồ thị thưa (Sparse Graphs Efficiency)

> **Đề bài:** Nếu định nghĩa đồ thị thưa (sparse graphs) là đồ thị thỏa mãn $|E| \in O(|V|)$, hãy cho biết cài đặt DFS nào có hiệu quả thời gian tốt hơn trên đồ thị thưa: cài đặt dùng ma trận kề hay cài đặt dùng danh sách kề?

> **Gợi ý của tác giả (Levitin):** So sánh các lớp hiệu quả thời gian của hai phiên bản DFS khi áp dụng cho đồ thị thưa.

Phân tích phương pháp giải:

Phân tích độ phức tạp thời gian tổng quát của DFS:

1. Dùng ma trận kề: Để tìm các đỉnh kề của mỗi đỉnh v, thuật toán phải duyệt qua toàn bộ một hàng kích thước $|V|$ trong ma trận. Với $|V|$ đỉnh, tổng thời gian duyệt là $\Theta(|V|^2)$, hoàn toàn không phụ thuộc vào số lượng cạnh $|E|$.

2. Dùng danh sách kề: Mỗi đỉnh được thăm đúng 1 lần, và danh sách kề của đỉnh v có độ dài $\deg(v)$. Tổng độ dài các danh sách kề trên toàn bộ đồ thị vô hướng là $\sum_{v \in V} \deg(v) = 2|E|$. Do đó tổng thời gian duyệt là $\Theta(|V| + |E|)$.

Lời giải chi tiết:

So sánh chi tiết:

1. **Cài đặt dùng ma trận kề (Adjacency Matrix):**

- Thời gian khởi tạo mảng đánh dấu: $\Theta(|V|)$.

- Với mỗi đỉnh được duyệt, thuật toán duyệt qua tất cả $|V|$ phần tử của hàng tương ứng để xác định các đỉnh kề:

$T_{\text{matrix}}(|V|, |E|) \in \Theta(|V|^2)$

- Độ phức tạp này luôn là bậc hai bất kể đồ thị có bao nhiêu cạnh.

2. **Cài đặt dùng danh sách kề (Adjacency Lists):**

- Thời gian thăm đỉnh: mỗi đỉnh được đánh dấu đúng 1 lần $\Longrightarrow \Theta(|V|)$.

- Thời gian duyệt cạnh: mỗi cạnh (u, v) được xét đúng 2 lần (một lần trong danh sách của u và một lần trong danh sách của v) $\Longrightarrow \Theta(|E|)$.

- Tổng thời gian:

$T_{\text{list}}(|V|, |E|) \in \Theta(|V| + |E|)$

3. **Đánh giá trên đồ thị thưa ($|E| \in O(|V|)$):**

- Thay $|E| \leq c \cdot |V|$ vào công thức danh sách kề:

$T_{\text{list}}(|V|, |E|) \in \Theta(|V| + O(|V|)) = \Theta(|V|)$

- Trong khi đó, với ma trận kề:

$T_{\text{matrix}}(|V|, |E|) \in \Theta(|V|^2)$

Kết luận:

Cài đặt dùng **danh sách kề (adjacency lists)** vượt trội hơn hẳn với thời gian **tuyến tính $\Theta(|V|)$**, trong khi cài đặt dùng ma trận kề tốn thời gian **bậc hai $\Theta(|V|^2)$**. Ví dụ, với mạng xã hội hoặc bản đồ đường phố có $|V| = 10^6$ đỉnh và mỗi đỉnh có trung bình 4 cạnh kề ($|E| =$

2×10^6), danh sách kề chỉ cần vài triệu phép tính (hoàn tất trong mili-giây), còn ma trận kề cần 10^{12} phép tính (mất nhiều giờ).

Bài 3.5.3: Bất biến cấu trúc của Rừng DFS (Invariants of DFS Forests)

> **Đề bài:** Cho đồ thị vô hướng G có n đỉnh và m cạnh.
> a. Đúng hay sai: Mọi rừng DFS của G (thu được từ các lượt duyệt bắt đầu ở các đỉnh khác nhau) đều có cùng số lượng cây?
> b. Đúng hay sai: Mọi rừng DFS của G đều có cùng số lượng cạnh cây (tree edges) và cùng số lượng cạnh ngược (back edges)?

> **Gợi ý của tác giả (Levitin):** a. Số lượng cây trong rừng DFS bằng bao nhiêu đại lượng topo của đồ thị? b. Trả lời câu hỏi cho đồ thị liên thông trước, sau đó tổng quát hóa.

Phân tích phương pháp giải:

Một khu rừng DFS (DFS forest) là tập hợp các cây DFS bao phủ tất cả các đỉnh của đồ thị. Ta cần phân tích mối liên hệ giữa các thành phần của rừng DFS với các bất biến topo của đồ thị G:

1. Tính liên thông và số thành phần liên thông c.

2. Một cây bao trùm (spanning tree) của một thành phần liên thông gồm k đỉnh luôn có đúng $k-1$ cạnh.

Lời giải chi tiết:

a. Câu khẳng định (a): ĐÚNG (TRUE).

Chứng minh:

- Mỗi lần thuật toán DFS bắt đầu duyệt từ một đỉnh chưa được thăm v, nó sẽ duyệt qua toàn bộ các đỉnh có thể đi tới được từ v thông qua các đường đi trong đồ thị vô hướng G.

- Tập hợp các đỉnh này chính là thành phần liên thông (connected component) chứa đỉnh v.

- Khi DFS duyệt xong thành phần liên thông này, cây DFS tương ứng kết thúc. Thuật toán chỉ khởi động một cây DFS mới khi còn đỉnh ngoài thành phần liên thông hiện tại chưa được thăm.

- Do đó, mỗi cây trong rừng DFS tương ứng 1-1 với một thành phần liên thông của đồ thị G.

- Vì số thành phần liên thông c của một đồ thị G là một bất biến topo cố định (không phụ thuộc vào thứ tự thăm đỉnh), nên mọi rừng DFS của G **luôn có đúng c cây**.

b. Câu khẳng định (b): ĐÚNG (TRUE).

Chứng minh:

- Giả sử đồ thị G có c thành phần liên thông $C_1, C_2, \dots, C_c$, trong đó thành phần thứ i có n_i đỉnh và m_i cạnh. Ta có:

$\sum_{i=1}^{c} n_i = n$ và $\sum_{i=1}^{c} m_i = m$

- Trong bất kỳ lượt duyệt DFS nào, cây DFS ứng với thành phần C_i là một cây bao trùm chứa toàn bộ n_i đỉnh của C_i. Theo tính chất cơ bản của cây, số cạnh cây của thành phần C_i luôn là:

$T_i = n_i - 1$

- Tổng số cạnh cây (tree edges) trong toàn bộ rừng DFS là:

$T_{\text{total}} = \sum_{i=1}^{c} T_i = \sum_{i=1}^{c} (n_i - 1) = \sum_{i=1}^{c} n_i - \sum_{i=1}^{c} 1 = n - c$

- Vì n (tổng số đỉnh) và c (số thành phần liên thông) là hằng số cố định của đồ thị G, số lượng cạnh cây $T_{\text{total}} = n - c$ luôn không đổi trong mọi rừng DFS.

- Trong đồ thị vô hướng, thuật toán DFS phân loại toàn bộ m cạnh của đồ thị thành hai loại rời nhau: cạnh cây (tree edges) và cạnh ngược (back edges). Do đó, số lượng cạnh ngược là:

$B_{\text{total}} = m - T_{\text{total}} = m - (n - c) = m - n + c$

- Biểu thức này phụ thuộc duy nhất vào m, n, c, hoàn toàn không phụ thuộc vào đỉnh xuất phát hay thứ tự chọn đỉnh kề.

Kết luận: Cả hai mệnh đề (a) và (b) đều **ĐÚNG**.

Bài 3.5.4: Duyệt BFS và dựng cây BFS (Breadth-First Search Trace)

> **Đề bài:** Duyệt đồ thị ở Bài 1 bằng thuật toán tìm kiếm theo chiều rộng (BFS) và dựng cây BFS tương ứng. Bắt đầu duyệt từ đỉnh a và ưu tiên thứ tự bảng chữ cái khi chọn đỉnh kề.

> **Gợi ý của tác giả (Levitin):** Thực hiện duyệt BFS từng bước sử dụng hàng đợi (FIFO Queue) tương tự Hình 3.11 trong sách giáo trình.

Phân tích phương pháp giải:

BFS sử dụng hàng đợi (Queue) FIFO:

1. Đỉnh gốc a được đưa vào hàng đợi ở Mức 0 (Level 0).

2. Lấy đỉnh u ở đầu hàng đợi ra, kiểm tra tất cả các đỉnh kề v chưa thăm của u theo thứ tự chữ cái, thêm v vào hàng đợi ở Mức tiếp theo và đánh dấu cạnh (u, v) là cạnh cây (tree edge).

3. Nếu gặp đỉnh kề v đã được thăm và không phải là cha trực tiếp, cạnh (u, v) là cạnh chéo (cross edge).

Lời giải chi tiết:

Các bước duyệt BFS từ đỉnh a:

1. **Khởi tạo:**

   - Hàng đợi: $Q = [a]$. Đánh dấu a đã thăm. Mức của a: $L(a) = 0$.

2. **Xử lý Mức 0 (đỉnh a):**

   - Lấy a ra khỏi Q. Các đỉnh kề của a theo thứ tự chữ cái: b, c, d, e (chưa đỉnh nào được thăm).

   - Thăm và đưa vào hàng đợi:

+ Thăm b, cạnh cây (a, b), $L(b) = 1$. Thêm b vào Q.

+ Thăm c, cạnh cây (a, c), $L(c) = 1$. Thêm c vào Q.

+ Thăm d, cạnh cây (a, d), $L(d) = 1$. Thêm d vào Q.

+ Thăm e, cạnh cây (a, e), $L(e) = 1$. Thêm e vào Q.

- Trạng thái hàng đợi sau Mức 0: $Q = [b, c, d, e]$.

3. **Xử lý Mức 1:**

- **Xét đỉnh b:** Lấy b ra khỏi Q. Các đỉnh kề: a (đã thăm), d (đã thăm $\Longrightarrow$ cạnh chéo (b, d)), f (chưa thăm).

+ Thăm f, cạnh cây (b, f), $L(f) = 2$. Thêm f vào Q.

+ Hàng đợi: $Q = [c, d, e, f]$.

- **Xét đỉnh c:** Lấy c ra khỏi Q. Các đỉnh kề: a (đã thăm), g (chưa thăm).

+ Thăm g, cạnh cây (c, g), $L(g) = 2$. Thêm g vào Q.

+ Hàng đợi: $Q = [d, e, f, g]$.

- **Xét đỉnh d:** Lấy d ra khỏi Q. Các đỉnh kề: a (đã thăm), b (đã xét), f (đã thăm $\Longrightarrow$ cạnh chéo (d, f)).

+ Không có đỉnh kề mới.

+ Hàng đợi: $Q = [e, f, g]$.

- **Xét đỉnh e:** Lấy e ra khỏi Q. Các đỉnh kề: a (đã thăm), g (đã thăm $\Longrightarrow$ cạnh chéo (e, g)).

+ Không có đỉnh kề mới.

+ Hàng đợi: $Q = [f, g]$.

4. **Xử lý Mức 2:**

- **Xét đỉnh f:** Lấy f ra khỏi Q. Các đỉnh kề: b, d (đều đã thăm).

- **Xét đỉnh g:** Lấy g ra khỏi Q. Các đỉnh kề: c, e (đều đã thăm).

- Hàng đợi rỗng: $Q = [\emptyset]$. Quá trình duyệt BFS kết thúc.

Cấu trúc Cây BFS:

- **Mức 0:** a (gốc cây)

- **Mức 1:** b, c, d, e (các con trực tiếp của a)

- **Mức 2:** f (con của b), g (con của c)

Phân loại các cạnh:

- **Tập cạnh cây (Tree edges, 6 cạnh):**

$\{(a, b), (a, c), (a, d), (a, e), (b, f), (c, g)\}$

- **Tập cạnh chéo (Cross edges, 3 cạnh):**

1. Cạnh (b, d) kết nối hai đỉnh cùng Mức 1.

2. Cạnh (d, f) kết nối đỉnh d (Mức 1) với đỉnh f (Mức 2).

3. Cạnh (e, g) kết nối đỉnh e (Mức 1) với đỉnh g (Mức 2).

Tổng số cạnh = $6 + 3 = 9$ cạnh.

Bài 3.5.5: Tính chất Mức của Cạnh chéo trong Cây BFS (Cross Edge Level Property)

> **Đề bài:** Chứng minh rằng trong cây BFS của một đồ thị vô hướng, một cạnh chéo (cross edge) chỉ có thể kết nối hai đỉnh nằm trên cùng một mức hoặc nằm trên hai mức kề nhau của cây BFS.

> **Gợi ý của tác giả (Levitin):** Sử dụng tính chất: Mức của một đỉnh u trong cây BFS biểu thị khoảng cách đường đi ngắn nhất (số lượng cạnh ít nhất) từ gốc đến đỉnh u.

Phân tích phương pháp giải:

Gọi r là gốc của cây BFS. Với mỗi đỉnh v, ký hiệu $L(v)$ là mức (level) của v trong cây BFS.

Định lý cốt lõi của BFS khẳng định: $L(v) = d(r, v)$, trong đó $d(r, v)$ là khoảng cách ngắn nhất (tính bằng số cạnh) từ r đến v.

Ta cần chứng minh: Với mọi cạnh $(u, v) \in E$, ta luôn có $|L(u) - L(v)| \leq 1$.

Lời giải chi tiết:

Chứng minh:

1. **Bổ đề khoảng cách ngắn nhất trong BFS:**

Trong đồ thị vô hướng không trọng số, thuật toán BFS thăm các đỉnh theo thứ tự tăng dần của khoảng cách từ gốc r. Do đó, mức $L(u)$ của đỉnh u trong cây BFS chính là độ dài đường đi ngắn nhất từ r đến u:

$L(u) = d(r, u)$

2. **Bất đẳng thức tam giác cho khoảng cách:**

Giả sử tồn tại cạnh $(u, v) \in E$. Khi đó, từ đường đi ngắn nhất từ r đến u, ta có thể đi thêm một cạnh (u, v) để tới v. Theo nguyên lý đường đi ngắn nhất:

$d(r, v) \leq d(r, u) + 1 \Leftrightarrow L(v) \leq L(u) + 1$

Hoàn toàn đối xứng, vì đồ thị là vô hướng nên cạnh $(v, u) \in E$, ta cũng có:

$d(r, u) \leq d(r, v) + 1 \Leftrightarrow L(u) \leq L(v) + 1 \Leftrightarrow L(v) \geq L(u) - 1$

3. **Kết hợp hai bất đẳng thức:**

Từ $L(u) - 1 \leq L(v) \leq L(u) + 1$, ta suy ra:

$-1 \leq L(v) - L(u) \leq 1 \Leftrightarrow |L(u) - L(v)| \leq 1$

4. **Ý nghĩa hình học trên cây BFS:**

Giá trị $|L(u) - L(v)|$ chỉ có thể nhận hai giá trị nguyên:

- $|L(u) - L(v)| = 0$: u và v ở **cùng một mức** ($L(u) = L(v)$).

- $|L(u) - L(v)| = 1$: u và v ở **hai mức kề nhau** ($L(u) = L(v) \pm 1$).

- Nếu cạnh (u, v) là cạnh cây (tree edge), thì u là cha của v (hoặc ngược lại), nên $|L(u) - L(v)| = 1$.

- Nếu cạnh (u, v) là cạnh chéo (cross edge), thì nó chỉ có thể nối hai đỉnh ở cùng mức hoặc hai đỉnh ở hai mức kề nhau (nhưng không có quan hệ cha-con).

Chứng minh bằng phản chứng dựa trên hoạt động của hàng đợi:

Giả sử ngược lại, tồn tại cạnh (u, v) sao cho $L(v) \geq L(u) + 2$.

Khi đỉnh u được lấy ra khỏi hàng đợi để duyệt ở Mức $L(u)$, thuật toán sẽ kiểm tra tất cả các đỉnh kề của nó:

- Nếu v chưa được thăm, v sẽ lập tức được đưa vào hàng đợi ở Mức $L(u) + 1$.

- Nếu v đã được thăm từ trước, thì v phải được đưa vào hàng đợi ở một mức nào đó $\leq L(u) + 1$.

Trong mọi trường hợp, v chắc chắn có mức $L(v) \leq L(u) + 1$, mâu thuẫn với giả thiết $L(v) \geq L(u) + 2$.

Do đó, cạnh chéo không bao giờ có thể bắc qua từ hai mức cách nhau từ 2 trở lên. (Điều phải chứng minh).

Bài 3.5.6: Kiểm tra tính không chu trình (Acyclicity Check) và So sánh Tốc độ DFS vs. BFS

> **Đề bài:** a. Giải thích cách kiểm tra tính không chu trình (acyclic) của một đồ thị bằng thuật toán tìm kiếm theo chiều rộng (BFS).
>
> b. Liệu một trong hai thuật toán duyệt—DFS hoặc BFS—có luôn tìm ra chu trình nhanh hơn thuật toán kia không? Nếu có, hãy chỉ ra thuật toán nào tốt hơn và giải thích tại sao; nếu không, hãy đưa ra hai ví dụ minh họa chứng minh điều đó.

> **Gợi ý của tác giả (Levitin):** a. Tính chất nào của rừng BFS báo hiệu sự hiện diện của chu trình? (Tương tự như rừng DFS). b. Câu trả lời là KHÔNG. Hãy tìm hai ví dụ chứng minh.

Phân tích phương pháp giải:

a. Trong đồ thị vô hướng, cây khung (spanning tree) không chứa chu trình. Bất kỳ cạnh nào không thuộc cây khung (cạnh ngược trong DFS hoặc cạnh chéo trong BFS) kết hợp với đường đi trên cây sẽ tạo thành một chu trình đơn.

b. Để chứng minh không có thuật toán nào luôn nhanh hơn thuật toán kia, ta cần xây dựng:

- Ví dụ 1: DFS phát hiện chu trình nhanh hơn BFS.

- Ví dụ 2: BFS phát hiện chu trình nhanh hơn DFS.

Lời giải chi tiết:

a. Phương pháp kiểm tra chu trình bằng BFS:

1. **Nguyên lý:**

- Trong quá trình duyệt BFS trên đồ thị vô hướng, khi đang duyệt đỉnh u, ta xét đỉnh kề v:

+ Nếu v đã được thăm và v **không phải là đỉnh cha trực tiếp** của u (tức là $v \neq \text{parent}[u]$), thì cạnh (u, v) là một **cạnh chéo (cross edge)**.

- Vì v đã được thăm, đã tồn tại một đường đi từ gốc r đến v trên cây BFS. Đồng thời cũng có đường đi từ r đến u. Hai đường đi này cùng với cạnh (u, v) tạo thành một **chu trình đơn** trong đồ thị.

2. **Quy tắc kết luận:**

- Đồ thị chứa chu trình ⟺ Rừng BFS xuất hiện ít nhất một **cạnh chéo (cross edge)**.

- Đồ thị không có chu trình (rừng/cây - acyclic) ⟺ Toàn bộ các cạnh được duyệt trong rừng BFS đều là **cạnh cây (tree edges)**, không có bất kỳ cạnh chéo nào.

b. So sánh tốc độ tìm chu trình giữa DFS và BFS:

Khẳng định: KHÔNG CÓ thuật toán nào luôn tìm ra chu trình nhanh hơn thuật toán kia trong mọi trường hợp.

Chứng minh bằng hai ví dụ đối lập:

- **Ví dụ 1 (DFS nhanh hơn BFS rõ rệt):**

+ **Cấu trúc đồ thị:** Giả sử đỉnh gốc r có 1000 đỉnh con ở Mức 1 $(v_1, v_2, \dots, v_{1000})$. Từ đỉnh v_1, có một nhánh đi sâu xuống gồm chu trình tam giác nhỏ $\{v_1, w_1, w_2, v_1\}$ nằm ngay ở bước kế tiếp. Còn lại 999 đỉnh kia mỗi đỉnh là gốc của một nhánh cây rất lớn.

+ **Hoạt động của DFS:** Bắt đầu từ r, đi xuống nhánh đầu tiên v_1, từ v_1 đi tới w_1, từ w_1 đi tới w_2, và gặp cạnh ngược (w_2, v_1). DFS phát hiện ra chu trình chỉ sau **4 bước** thăm đỉnh!

+ **Hoạt động của BFS:** Bắt đầu từ r, BFS bắt buộc phải thăm và đưa toàn bộ 1000 đỉnh con ở Mức 1 vào hàng đợi trước khi có thể mở rộng xuống Mức 2. BFS tốn hơn 1000 phép tính duyệt mới chạm tới chu trình.

+ ⟹ Ở trường hợp này, DFS nhanh hơn BFS hàng trăm lần.

- **Ví dụ 2 (BFS nhanh hơn DFS rõ rệt):**

+ **Cấu trúc đồ thị:** Đỉnh gốc r kết nối trực tiếp với một chu trình tam giác ngay ở Mức 1: cạnh $(r, u), (r, v)$ và cạnh (u, v). Đồng thời, đỉnh r cũng có một cạnh nối với đỉnh w, và từ w mọc ra một đường đi cực dài (hoặc một cây khổng lồ) không chứa chu trình với 10^6 đỉnh. Giả sử theo thứ tự bảng chữ cái hoặc thứ tự danh sách kề, nhánh w được xét trước nhánh u, v.

+ **Hoạt động của DFS:** DFS chọn đi theo nhánh w, và vì DFS đi sâu không ngừng, nó sẽ duyệt toàn bộ 10^6 đỉnh của cây khổng lồ này, đi tới tận cùng ngõ cụt rồi mới quay lui về r để xét nhánh chứa chu trình. DFS tốn $\Theta(10^6)$ phép tính.

+ **Hoạt động của BFS:** BFS duyệt theo chiều rộng từng mức. Ở Mức 1, BFS thăm w, u, v. Sau đó ngay khi lấy u ra duyệt, BFS gặp cạnh (u, v) nối tới v (đã thăm ở cùng Mức 1) ⟹ phát hiện chu trình ngay lập tức chỉ sau **vài bước**.

+ ⟹ Ở trường hợp này, BFS nhanh hơn DFS hàng triệu lần.

Tổng kết: Tốc độ phát hiện chu trình phụ thuộc hoàn toàn vào cấu trúc hình học của đồ thị và vị trí của chu trình (nằm nông gần gốc thì BFS lợi thế, nằm sâu theo một nhánh đơn thì DFS lợi thế).

Bài 3.5.7: Thuật toán Xác định Các Thành phần Liên thông (Connected Components)

> **Đề bài:** Giải thích cách xác định các thành phần liên thông của một đồ thị bằng cách sử dụng:
> a. Tìm kiếm theo chiều sâu (DFS).
> b. Tìm kiếm theo chiều rộng (BFS).

> **Gợi ý của tác giả (Levitin):** Cả hai phép duyệt đều chỉ có thể tới được đỉnh mới nếu đỉnh đó kề với một trong các đỉnh đã thăm. Hỏi những đỉnh nào sẽ được thăm khi lượt duyệt hiện tại kết thúc (khi ngăn xếp hoặc hàng đợi rỗng)?

Phân tích phương pháp giải:

Một thành phần liên thông của đồ thị vô hướng $G = (V, E)$ là một đồ thị con liên thông cực đại.

Bắt đầu từ một đỉnh bất kỳ v, một lượt duyệt hoàn chỉnh (dù là DFS hay BFS) sẽ thăm chính xác tất cả các đỉnh thuộc cùng thành phần liên thông chứa v.

Bằng cách lặp qua tất cả các đỉnh từ 1 đến n, mỗi khi gặp một đỉnh chưa được gán nhãn thành phần, ta khởi động một lượt duyệt mới và tăng biến đếm thành phần liên thông.

Lời giải chi tiết:

a. Xác định thành phần liên thông bằng DFS:

1. **Ý tưởng:**

- Duy trì một mảng `component_id` kích thước $|V|$, khởi tạo tất cả bằng 0 (chưa thăm).
- Biến đếm số thành phần liên thông `count = 0`.
- Lặp qua từng đỉnh $v \in V$:
  + Nếu `component_id[v] == 0`, tăng `count = count + 1`.
  + Gọi hàm đệ quy `DFS_Component(v, count)`: hàm này sẽ duyệt toàn bộ các đỉnh có thể tới được từ v và gán `component_id[u] = count` cho mỗi đỉnh u được thăm.
- Khi vòng lặp kết thúc, `count` chính là tổng số thành phần liên thông, và các đỉnh có cùng giá trị `component_id` thuộc về cùng một thành phần liên thông.

2. **Cấu trúc mã giả chuẩn hóa:**

```
function: ConnectedComponentsDFS(G): Map<Integer, Set<Vertex>>
    input: G = (V, E): Graph – undirected graph with vertices V and edges E.
    output: components: Map<Integer, Set<Vertex>> – mapping component ID to vertex sets.
 1  count = 0
```

```
 2  for each vertex v in V do
 3      component[v] = 0 – 0 denotes unvisited.
 4  end-for each
 5  for each vertex v in V do
 6      if component[v] = 0 then
 7          count = count + 1
 8          DFS_Mark(v, count)
 9      end-if
10  end-for each
11  return component.
function: DFS_Mark(v, id): void
    input: v: Vertex – starting vertex.
           id: Integer – component identifier.
 1  component[v] = id – assign component id.
 2  for each neighbor w of v do
 3      if component[w] = 0 then
 4          DFS_Mark(w, id)
 5      end-if
 6  end-for each
```

b. Xác định thành phần liên thông bằng BFS:

1. **Ý tưởng:**

- Tương tự như DFS, nhưng thay vì đệ quy, ta dùng hàng đợi FIFO (Queue).
- Với mỗi đỉnh v chưa thăm, tăng `count = count + 1`, đẩy v vào hàng đợi `Q`, gán `component[v] = count`.
- Lấy từng đỉnh u khỏi `Q`, với mỗi đỉnh kề w chưa thăm (`component[w] == 0`), gán `component[w] = count` và đẩy w vào `Q`.
- Khi `Q` rỗng, toàn bộ thành phần liên thông thứ `count` đã được gom đủ.

2. **Cấu trúc mã giả chuẩn hóa:**

```
function: ConnectedComponentsBFS(G): Map<Integer, Set<Vertex>>
    input: G = (V, E): Graph – undirected graph.
    output: components: Map<Integer, Set<Vertex>> – component partitions.
 1  count = 0
```

```
 2  for each vertex v in V do
 3      component[v] = 0
 4  end-for each
 5  for each vertex v in V do
 6      if component[v] = 0 then
 7          count = count + 1
 8          Q = empty queue
 9          component[v] = count
10           Enqueue(Q, v)
11           while not IsEmpty(Q) do
12               u = Dequeue(Q)
13               for each neighbor w of u do
14                   if component[w] = 0 then
15                       component[w] = count
16                       Enqueue(Q, w)
17                   end-if
18               end-for each
19           end-while
20      end-if
21  end-for each
22  return component.
```

Độ phức tạp: Cả hai thuật toán đều có độ phức tạp thời gian tối ưu $\Theta(|V| + |E|)$ khi dùng danh sách kề.

Bài 3.5.8: Thuật toán Kiểm tra Đồ thị Hai phía (Bipartite / 2-Colorable Graph Checking)

> **Đề bài:** Một đồ thị được gọi là hai phía (bipartite) nếu tất cả các đỉnh của nó có thể phân hoạch thành hai tập con rời nhau X và Y sao cho mọi cạnh đều nối một đỉnh trong X với một đỉnh trong Y (tương đương với việc có thể tô màu các đỉnh bằng 2 màu sao cho không có hai đỉnh kề nào cùng màu; còn gọi là đồ thị 2-tô-màu được).
> a. Thiết kế thuật toán dựa trên DFS để kiểm tra xem một đồ thị có phải là hai phía hay không.
> b. Thiết kế thuật toán dựa trên BFS để kiểm tra xem một đồ thị có phải là hai phía hay không.

> **Gợi ý của tác giả (Levitin):** a. Sử dụng rừng DFS: kiểm tra xem có cạnh ngược nào nối hai đỉnh cùng

> màu hay không. b. Sử dụng rừng BFS: kiểm tra xem có cạnh chéo nào nối hai đỉnh cùng mức hay không.

Phân tích phương pháp giải:

Định lý König (1936): Một đồ thị là hai phía khi và chỉ khi nó KHÔNG chứa bất kỳ chu trình lẻ nào (chu trình có số cạnh lẻ).

Phương pháp tô 2 màu (màu 1 và màu 2):

- Bắt đầu từ một đỉnh, tô màu 1.

- Tất cả các đỉnh kề trực tiếp phải được tô màu đối lập: màu 2.

- Trong DFS: Nếu gặp một đỉnh kề *w* đã được thăm mà `color[w] == color[v]`, ta phát hiện một chu trình lẻ ⟹ đồ thị không phải là hai phía.

- Trong BFS: Các mức chẵn mang màu 1, các mức lẻ mang màu 2. Cạnh chéo giữa hai đỉnh cùng mức nối hai đỉnh cùng màu ⟹ tạo thành chu trình lẻ.

Lời giải chi tiết:

a. Thuật toán kiểm tra đồ thị hai phía dựa trên DFS:

```
function: IsBipartiteDFS(G): Boolean
    input: G = (V, E): Graph – undirected graph.
    output: true if G is bipartite, false otherwise.
 1  for each vertex v in V do
 2      color[v] = 0 – 0 means uncolored.
 3  end-for each
 4  for each vertex v in V do
 5      if color[v] = 0 then
 6          if not DFS_CheckBipartite(v, 1) then
 7              return false – odd cycle detected.
 8          end-if
 9      end-if
10  end-for each
11  return true – graph is 2-colorable.
function: DFS_CheckBipartite(v, c): Boolean
    input: v: Vertex – current vertex.
           c: Integer – color to assign (1 or 2).
    output: true if valid 2-coloring exists for this component, false otherwise.
```

```
 1  color[v] = c
 2  next_color = 3 - c – toggles between 1 and 2.
 3  for each neighbor w of v do
 4     if color[w] = 0 then
 5        if not DFS_CheckBipartite(w, next_color) then
 6           return false
 7        end-if
 8     else if color[w] = color[v] then
 9        return false – conflict: adjacent vertices share color.
10     end-if
11  end-for each
12  return true.
```

b. Thuật toán kiểm tra đồ thị hai phía dựa trên BFS:

```
function: IsBipartiteBFS(G): Boolean
   input: G = (V, E): Graph – undirected graph.
   output: true if G is bipartite, false otherwise.
 1  for each vertex v in V do
 2     color[v] = 0 – 0 denotes uncolored.
 3  end-for each
 4  for each vertex v in V do
 5     if color[v] = 0 then
 6        Q = empty queue
 7        color[v] = 1 – color root with 1.
 8        Enqueue(Q, v)
 9        while not IsEmpty(Q) do
10           u = Dequeue(Q)
11           for each neighbor w of u do
12              if color[w] = 0 then
13                 color[w] = 3 - color[u] – alternate color.
14                 Enqueue(Q, w)
15              else if color[w] = color[u] then
16                 return false – cross edge between vertices of same color.
```

```
17                end-if
18             end-for each
19          end-while
20       end-if
21    end-for each
22    return true – graph is bipartite.
```

Độ phức tạp: Cả hai thuật toán đều chạy trong thời gian $\Theta(|V| + |E|)$ và bộ nhớ phụ trợ $\Theta(|V|)$.

Bài 3.5.9: Thiết kế Chương trình Xuất Thành phần liên thông và Chu trình (Cycle Reconstruction Program)

> **Đề bài:** Viết một chương trình (thuật toán) nhận vào một đồ thị và xuất ra:
> a. Danh sách các đỉnh của từng thành phần liên thông.
> b. Một chu trình cụ thể của đồ thị (nếu có) dưới dạng dãy các đỉnh liên tiếp, hoặc đưa ra thông báo đồ thị không có chu trình (acyclic).

> **Gợi ý của tác giả (Levitin):** Sử dụng DFS hoặc BFS. Khi phát hiện một cạnh ngược (u, v) trong DFS (hoặc cạnh chéo trong BFS), sử dụng mảng con trỏ cha (parent) để truy vết ngược lại đường đi từ u về v, từ đó dựng nên chu trình đơn.

Phân tích phương pháp giải:

Thiết kế thuật toán tối ưu dựa trên DFS:

1. Thành phần liên thông: Gom nhóm các đỉnh trong mỗi lượt gọi hàm DFS gốc.

2. Truy vết chu trình: Duy trì mảng `parent[u]` ghi lại đỉnh cha trực tiếp của u trên cây DFS.

Khi xét đỉnh kề w của u:

- Nếu w đã được thăm và $w \neq parent[u]$, ta vừa phát hiện một cạnh ngược (u, w).

- Cạnh này tạo thành chu trình với đường đi trên cây từ w xuống u.

- Truy vết: bắt đầu từ u, lùi dần qua các đỉnh cha $parent[u], parent[parent[u]], \ldots$ cho đến khi chạm tới w, sau đó khép vòng lại về u.

Lời giải chi tiết:

Cấu trúc thuật toán tổng thể:

```
function: AnalyzeGraph(G): void
   input: G = (V, E): Graph – undirected graph.
```

```
   global: visited: Array[1..n] of Boolean
           parent: Array[1..n] of Vertex
           cycle_found: Boolean
           cycle_path: List of Vertex
   output: prints components and detected cycle (if any).
 1  for each v in V do
 2      visited[v] = false
 3      parent[v] = null
 4  end-for each
 5  cycle_found = false
 6  comp_id = 0
 7  for each v in V do
 8      if not visited[v] then
 9          comp_id = comp_id + 1
10          comp_vertices = empty list
11          DFS_Explore(v, comp_vertices)
12          Print("Thanh phan lien thong ", comp_id, ": ", comp_vertices)
13      end-if
14  end-for each
15  if cycle_found then
16      Print("Do thi chua chu trinh: ", cycle_path)
17  else
18      Print("Do thi khong chua chu trinh (Acyclic/Forest).")
19  end-if
20  return.
function: DFS_Explore(u, comp_list): void
   input: u: Vertex – current vertex.
          comp_list: List – accumulates vertices in current component.
 1  visited[u] = true
 2  Append(comp_list, u)
 3  for each neighbor w of u do
 4      if not visited[w] then
 5          parent[w] = u
 6          DFS_Explore(w, comp_list)
```

```
7     else if w != parent[u] and not cycle_found then
8        – Back edge (u, w) detected: reconstruct cycle from u to w.
9        cycle_found = true
10       curr = u
11       cycle_path = empty list
12       while curr != w do
13          Append(cycle_path, curr)
14          curr = parent[curr]
15       end-while
16       Append(cycle_path, w)
17       Append(cycle_path, u) – close cycle loop.
18    end-if
19 end-for each
20 return.
```

Giải thích chi tiết hoạt động:

1. Phép duyệt duyệt qua mọi đỉnh và cạnh đúng một lần $\Longrightarrow$ Thời gian chạy $O(|V| + |E|)$.

2. Khi gặp cạnh ngược (u, w), vòng lặp `while` truy vết từ u ngược theo con trỏ `parent` lên tổ tiên w. Vì w là tổ tiên trực tiếp của u trên cây DFS, độ dài đường đi không vượt quá chiều cao cây ($O(|V|)$).

3. Danh sách `cycle_path` thu được chính là chu trình đơn: $u \to parent[u] \to \cdots \to w \to u$.

Bài 3.5.10: Mô hình hóa Mê cung bằng Đồ thị và Chiến lược Thoát hiểm (Maze Modeling & Exploration)

> **Đề bài:** Ta có thể mô hình hóa một mê cung bằng đồ thị: tạo một đỉnh cho điểm xuất phát (starting point), điểm kết thúc (finishing point), các ngõ cụt (dead ends), và tất cả các giao lộ trong mê cung (nơi có từ 2 hướng rẽ trở lên), sau đó nối các đỉnh bằng các cạnh tương ứng với các hành lang trong mê cung.
>
> a. Hãy xây dựng đồ thị mô hình hóa cho mê cung được minh họa trong sách giáo trình.
>
> b. Nếu bạn thấy mình bị lạc trong một mê cung ngoài đời thực, bạn sẽ sử dụng thuật toán duyệt nào—DFS hay BFS—và tại sao?

> **Gợi ý của tác giả (Levitin):** a. Làm theo đúng định nghĩa các đỉnh trong phát biểu bài toán. b. Hãy thử tưởng tượng việc thực hiện từng thuật toán bằng cách đi bộ trong thực tế.

Phân tích phương pháp giải:

a. Trong mô hình đồ thị mê cung:

- Các hành lang thẳng không có ngã rẽ chỉ là đường nối giữa hai đỉnh quan trọng.

- Đỉnh là: Điểm xuất phát (S), Đích (F), các ngã ba/ngã tư (Intersections), và ngõ cụt (Dead ends).

b. Đây là câu hỏi kinh điển về sự khác biệt giữa mô hình tính toán lý thuyết và hạn chế vật lý của con người:

- BFS đòi hỏi phải thăm TẤT CẢ các đỉnh ở cự ly 1 bước, sau đó thăm TẤT CẢ các đỉnh ở cự ly 2 bước...

- DFS đi theo một lối đi duy nhất cho đến khi chạm ngõ cụt, sau đó quay lui từng bước về giao lộ gần nhất.

Lời giải chi tiết:

a. Mô hình hóa mê cung bằng đồ thị $G = (V, E)$:

1. **Tập đỉnh V:**

- Đỉnh S: Điểm xuất phát (Start / Entrance).

- Đỉnh F: Điểm thoát hiểm (Exit / Finish).

- Các đỉnh ngã rẽ $J_1, J_2, \dots, J_k$: Các điểm giao nhau nơi hành lang phân tách thành từ 2 hướng trở lên.

- Các đỉnh ngõ cụt $D_1, D_2, \dots, D_m$: Các điểm cuối của hành lang bị bít kín.

2. **Tập cạnh E:**

- Mỗi cạnh $(u, v) \in E$ tương ứng với một đoạn hành lang liên tục nối trực tiếp hai điểm nút u và v mà không bị gián đoạn bởi bất kỳ ngã rẽ nào khác.

- Đồ thị thu được là một đồ thị phẳng vô hướng (planar graph). Đường đi thoát khỏi mê cung tương ứng với một đường đi đơn từ đỉnh S đến đỉnh F.

b. Lựa chọn thuật toán khi bị lạc trong mê cung ngoài đời thực:

Lựa chọn dứt khoát: Ta **BẮT BUỘC PHẢI DÙNG DFS (Tìm kiếm theo chiều sâu)**.

Lý do khoa học và thực tế:

1. **Khả thi về mặt vật lý (Physical Feasibility):**

- Thuật toán **DFS tương ứng chính xác với việc một con người đi bộ liên tục**: ta đi dọc theo một hành lang, nếu tới ngã rẽ thì chọn một hướng đi tiếp; nếu gặp ngõ cụt, ta chỉ việc quay đầu bước lùi lại ngã rẽ gần nhất vừa đi qua để thử hướng khác.

- Để hiện thực hóa DFS trong mê cung, con người đã có phương pháp kinh điển từ thời cổ đại: **Thuật toán Trémaux** (dùng phấn đánh dấu lối đi) hoặc **Quy tắc tay phải / tay trái** (luôn áp một tay vào tường mê cung và đi tiếp mà không bao giờ rời tay), hoặc dùng **Cuộn chỉ Ariadne** như thần thoại Hy Lạp.

2. **Sự bất khả thi của BFS ngoài đời thực (The Infeasibility of BFS):**

- Thuật toán BFS yêu cầu duyệt theo từng mức đồng tâm: trước tiên khám phá tất cả các hành lang cách điểm xuất phát 10 mét, sau đó khám phá tất cả các hành lang cách 20 mét...

- Một con người không thể có khả năng 'dịch chuyển tức thời' (teleport) từ nhánh mê cung phía Đông sang nhánh mê cung phía Tây để duy trì hàng đợi FIFO của BFS.

- Nếu cố tình thực hiện BFS, mỗi lần đổi sang một đỉnh khác trong hàng đợi, người đó sẽ phải đi bộ một quãng đường khổng lồ quay lại gốc rồi đi sang nhánh khác, dẫn đến quãng đường di chuyển vật lý tăng theo cấp số nhân, gây kiệt sức và bất khả thi.

Bài 3.5.11: Bài toán Ba Chiếc Bình của Siméon Poisson (Three Jugs Puzzle solved by BFS)

> **Đề bài:** Nhà toán học và vật lý học nổi tiếng người Pháp Siméon Denis Poisson (1781–1840) được cho là đã nảy sinh niềm đam mê toán học sau khi giải câu đố cổ điển sau đây:
> Cho ba chiếc bình có dung tích lần lượt là 8 panh (pint), 5 panh và 3 panh. Ban đầu bình 8 panh chứa đầy nước, hai bình 5 panh và 3 panh hoàn toàn rỗng. Bằng các thao tác rót nước qua lại giữa các bình (chỉ được rót cho đến khi bình rót cạn hoặc bình nhận đầy), hãy đong được chính xác 4 panh nước vào một trong các bình.
> Hãy giải câu đố này bằng thuật toán tìm kiếm theo chiều rộng (BFS).

> **Gợi ý của tác giả (Levitin):** Bạn có thể áp dụng BFS mà không cần phải vẽ toàn bộ đồ thị trước; hãy sinh trạng thái theo từng mức của BFS.

Phân tích phương pháp giải:

Mô hình hóa bài toán không gian trạng thái (State Space Graph):

1. Trạng thái: Bộ ba số nguyên (x, y, z) biểu thị lượng nước hiện có trong ba bình:

   $x \in [0,8]$ (bình 8), $y \in [0,5]$ (bình 5), $z \in [0,3]$ (bình 3).

   Định luật bảo toàn khối lượng: $x + y + z = 8$ tại mọi thời điểm.

2. Trạng thái xuất phát (Root): $S_0 = (8,0,0)$.

3. Trạng thái đích (Goal): Bất kỳ trạng thái nào có $x = 4$ hoặc $y = 4$ (ví dụ: $(4, 4, 0)$).

4. Thao tác rót (Transitions): Có 6 thao tác rót từ bình i sang bình j $(i, j \in \{1,2,3\}, i \neq j)$:

   Lượng nước rót là $\Delta = min($nước bình i,dung tích còn lại của bình $j)$.

5. Áp dụng BFS: BFS đảm bảo tìm ra chuỗi thao tác rót ít bước nhất (Shortest Path).

Lời giải chi tiết:

Quá trình duyệt BFS theo từng Mức (Level-by-Level Trace):

- **Mức 0 (Trạng thái xuất phát):**

+ $(8, 0, 0)$

- **Mức 1 (Các trạng thái sinh ra từ Mức 0):**

+ Rót 8 → 5: **$(3, 5, 0)$**

+ Rót 8 → 3: **$(5, 0, 3)$**

- **Mức 2:**

+ Từ $(3, 5, 0)$:

- Rót $5 \rightarrow 3$: **$(3, 2, 3)$**
- Rót $5 \rightarrow 8$: $(8, 0, 0)$ (đã thăm)

+ Từ $(5, 0, 3)$:

- Rót $3 \rightarrow 5$: **$(5, 3, 0)$**
- Rót $3 \rightarrow 8$: $(8, 0, 0)$ (đã thăm)
- **Mức 3:**

+ Từ $(3, 2, 3)$:

- Rót $3 \rightarrow 8$: **$(6, 2, 0)$**
- Rót $2 \rightarrow 8$: $(5, 0, 3)$ (đã thăm)

+ Từ $(5, 3, 0)$:

- Rót $5 \rightarrow 8$: $(8, 0, 0)$ (đã thăm)
- Rót $5 \rightarrow 3$: $(5, 0, 3)$ (đã thăm)
- Rót $8 \rightarrow 3$: **$(2, 3, 3)$**
- **Mức 4:**

+ Từ $(6, 2, 0)$:

- Rót $5 \rightarrow 3$: **$(6, 0, 2)$** (rót hết 2 panh từ bình 5 sang bình 3)

+ Từ $(2, 3, 3)$:

- Rót $3 \rightarrow 5$: **$(2, 5, 1)$**
- **Mức 5:**

+ Từ $(6, 0, 2)$:

- Rót $8 \rightarrow 5$: **$(1, 5, 2)$**

+ Từ $(2, 5, 1)$:

- Rót $5 \rightarrow 8$: **$(7, 0, 1)$**
- **Mức 6:**

+ Từ $(1, 5, 2)$:

- Rót từ bình 5 sang bình 3 (bình 3 đang có 2 panh, chỉ nhận thêm 1 panh nữa để đầy 3 panh):
- Lượng nước còn lại trong bình 5 chính là: $5 - 1 = 4$ panh!
- Trạng thái mới: **$(1, 4, 3)$ $\Longrightarrow$ ĐẠT MỤC TIÊU (bình 5 có đúng 4 panh)!**

+ Nếu muốn chia đều 4 panh vào cả hai bình 8 và 5, ta thực hiện thêm 1 bước nữa ở Mức 7:

- Từ $(1, 4, 3)$, rót toàn bộ 3 panh từ bình 3 sang bình 8 $\Longrightarrow$ **$(4, 4, 0)$**!

Bảng tổng kết chuỗi hành động tối ưu (Đường đi ngắn nhất tìm bởi BFS):

Bước	Thao tác rót	Bình 8 (8p)	Bình 5 (5p)	Bình 3 (3p)
0	Trạng thái ban đầu	8	0	0
1	Rót từ bình 8 sang bình 5	3	5	0
2	Rót từ bình 5 sang bình 3	3	2	3
3	Rót từ bình 3 sang bình 8	6	2	0
4	Rót từ bình 5 sang bình 3	6	0	2
5	Rót từ bình 8 sang bình 5	1	5	2
6	Rót từ bình 5 sang bình 3 (cho đầy)	1	**4**	3
7	Rót từ bình 3 sang bình 8	**4**	**4**	0

Kết luận:

Thuật toán BFS đã tìm ra phương án tối ưu nhất:

- Chỉ mất đúng **6 bước rót** để thu được 4 panh nước đầu tiên (nằm trong bình 5 tại trạng thái $(1, 4, 3)$).

- Hoặc đúng **7 bước rót** để chia đôi lượng nước hoàn hảo thành hai phần bằng nhau $(4, 4, 0)$.

TỔNG KẾT TOÀN DIỆN CHƯƠNG 3 (SUMMARY & BẢNG TRA CỨU ĐỘ PHỨC TẠP)

1. **Bản chất của Chiến lược Vét cạn (Brute Force):**

- Là phương pháp tiếp cận trực diện, bám sát phát biểu bài toán và các định nghĩa cơ bản.

- Điểm mạnh then chốt: Tính áp dụng rộng rãi, dễ hiểu, dễ cài đặt, là nền tảng để kiểm chứng tính đúng đắn của các thuật toán phức tạp hơn.

- Điểm yếu: Hiệu quả thời gian thường ở mức trung bình đến kém (thường là $O(n^2), O(n^3)$ hoặc hàm mũ $O(2^n), O(n!)$).

2. **Tìm kiếm Toàn diện (Exhaustive Search):**

- Là dạng vét cạn đặc thù áp dụng cho các bài toán tối ưu hóa tổ hợp (TSP, Cái Balo, Phân việc).

- Sinh toàn bộ không gian nghiệm khả dĩ (tập con 2^n hoặc hoán vị $n!$), lọc các phương án thỏa mãn ràng buộc và tìm phương án tối ưu.

- Bùng nổ tổ hợp khiến phương pháp này bất khả thi với dữ liệu lớn, đòi hỏi các kỹ thuật nâng cao hơn như Quy hoạch động, Nhánh cận hoặc Thuật toán xấp xỉ.

3. **Duyệt đồ thị (Graph Traversals - DFS & BFS):**

- Hai trụ cột của lý thuyết đồ thị với độ phức tạp thời gian tối ưu $\Theta(|V| + |E|)$ khi dùng danh sách kề.

- DFS dùng ngăn xếp (Stack/LIFO), hữu ích cho phát hiện chu trình, sắp xếp topo, tìm thành phần liên thông mạnh.

- BFS dùng hàng đợi (Queue/FIFO), tối ưu cho tìm đường đi ngắn nhất (ít cạnh nhất) và kiểm tra tính 2 phía (bipartite).